



1. Kickstarter for Beginners

This section is for Java beginners. It does not directly relate to any exam objective but is meant to provide a solid grounding that will help you to easily understand the concepts taught in later chapters. The concepts covered in this section are important because they repeat over and over throughout this book. If we get these repetitions over with now, you will be happier later on!

1.1 Programming and Programming Languages

We receive "instructions" all the time. When we are kids, we receive instructions from our parents. When we are in school, we receive instructions from our teachers. When we grow up, we receive instructions from our team leads, managers, and CEOs, customers, employers, and so on. One obvious fact about receiving instructions, which is very important in this discussion, is that we receive instructions in the language that we understand. Indeed, why would anyone give us instructions in a language that we don't understand? Most of the time, we follow the instructions and we do as we are told. But sometimes, we don't follow them. Either because we don't understand them, or because we are not able to do them. Well, sometimes we just don't want to do them but let's leave that case for now :)

In the computer world, the term used to describe the act of providing instructions is "programming" or "coding" and the set of instructions is called a "program" or the "code". Thus, when parents instruct children on how to behave, you could say that parents are "programming" the children, or when a teacher gives instructions on how to answer a test, you could say that the teacher is "programming" the students. If the language of instruction in the above situations is English, then English is the "programming language"!

The relation between a computer and a programmer is very similar. A programmer instructs a computer to do something in a language that the computer understands. In other words, a programmer programs a computer using a "programming language", which the computer understands. If the instructions given by the programmer are understood by the computer and are within the capabilities of the computer, then the computer will "execute" them and the outcome will be as the programmer expected. If you have ever followed a recipe from a cookbook, you know what I am talking about. You are being programmed by the author to produce the dish. From your perspective, you are simply executing the instructions given in the recipe. You trust the author to give you the right instructions and, upon executing those instructions exactly as given, you can expect the dish as the outcome.

But the fact remains that computers are not humans. They are just electronic machines with a very limited set of things that they can do. Furthermore, the only "language" that computers understand is "machine language", the language that has only two alphabets - 0 and 1, and which few humans can converse in easily. So, how are most people going to interact with a machine whose language they can't speak? This situation is no different from the one where you go to a restaurant in a foreign country and you don't speak the language that the waiter understands. That's where a translator (also known as an interpreter) comes into the picture. A translator understands two languages - your language and the waiter's language. He translates your instructions into the language that the waiter understands. Similarly, to help those who can't speak machine language, the people who understand machine language create "translators", which convert instructions written in a more human understandable language into instructions in the machine language. The technical terms for such translators are "compilers" and "interpreters". There are subtle differences between a compiler and an interpreter but you don't need to worry about that right now.

The languages that most programmers are familiar with and are able to write the instructions in are called "high level languages". You must have heard of Java, C/C++, Python, and JavaScript. They are all high level languages. They are called high level because they use words from a human language (English) and are a lot easier to read and write for humans. But they cannot be understood

by a machine. The instructions written in a high level language have to be translated into a "low level language", which the computer can understand.

The following figure summarizes the above discussion. You have a programmer, who writes instructions in a high level language, a translator, which converts the instructions in a machine language, and a computer that executes the instructions and produces the output expected by the programmer.

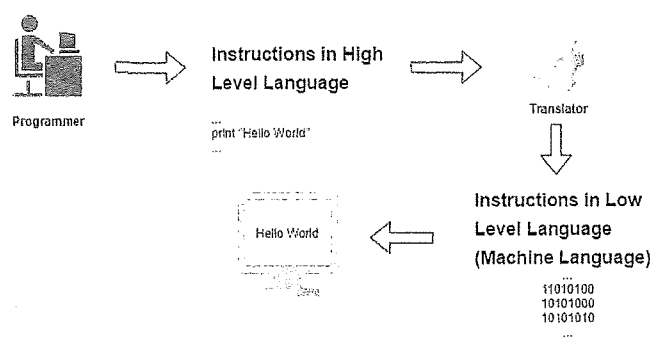


Figure 1.1.1: Steps in writing and executing a program

The above diagram illustrates the basic steps in programming and execution. However, this diagram doesn't exactly match what we observe in real life. Specifically, the steps where the programmer writes a program, and translates it using a translator, do not happen outside the computer. We see everything happening in the computer itself and that causes confusion in students who are just beginning to learn programming.

A long long time ago, in 1950s, and 60s, these steps used to occur outside a computer. At that time a computer was just a simple microprocessor chip stuck on circuit board. There was no screen or even a keyboard. A programmer would write the code directly in the machine language by punching holes on a piece of paper cards and then feed those cards through a special device attached to the microprocessor. Later on, microprocessors became a little more powerful and started supporting monitors and keyboards. The programmers would then feed the code into the computer using keyboards. This is when the microprocessors based devices started to look like the computers that you see today. So, while a microprocessor is still the heart of a computer, you need other peripheral devices such as memory bank, a monitor and a keyboard to harness the power of the microprocessor. If you open up your phone, laptop, or computer cover, you will see a board on which a lot of chips as well as devices are stuck. In the old times, this board and the chips used to be a lot larger in size but the basic architecture of the computer has remained the same.

Very soon, the execution power of computers increased due to advancement in electronics and people started using computers to do a variety of things ranging from word processing to data management. The set of instructions required to program a computer to do these things increased exponentially and it was practically impossible to write so much of machine language code and feed it to the computer manually. This led programmers to write helper programs that would accept instructions in a high level language and convert them into the machine language. These helper programs are called "compilers" and "interpreters". Similarly, they wrote helper programs that would take the machine level code and feed it to the microprocessor. The program that manages

the microprocessor and the devices connected to the microprocessor is called "Operating System". Windows, Android, Linux, MacOS, are all operating systems.

So, instead of dealing with the microprocessor directly, programmers take help of these helper programs to interact with the microprocessor. Both - compilers/interpreters, and the Operating System, reside in the memory of the computer itself. In other words, the basic steps shown in the above diagram still happen, but they performed by the programs running inside the computer.

1.2 Applications

In its early days, programming involved writing all the instructions for achieving a particular task in a file and submitting that file to the operating system for execution. As tasks became more and more varied and complicated, programs became too huge to maintain in a single file. Programmers started organizing their code into multiple files for ease of management and also so that they could reuse parts of it for performing other tasks. For example, if a program that printed account details were split into two separate parts where one part would just do the account management and the second part would just do the printing, then the printing part could be reused in another program that printed salary details. Such reusable parts are packaged separately and are called "libraries". A program, therefore, is composed of new code as well existing code present in a library.

With the advent of powerful operating systems such as Unix and Windows, programmers started writing programs for people who had no idea about programming but who could use the power of computers for performing their routine tasks such as writing documents and letters, maintaining their daily expenses, tracking their investments, and so on. Such ready to use programs are called "applications" or "apps", in the world of smartphones and portable devices. Microsoft Word, PowerPoint, and Google Chrome browser are examples of commonly used applications. You need to have these applications "installed" on your device before you can use them. Once installed, you can ask the operating system to execute that application, usually by clicking on an icon. You can also execute it by specifying the name of the application on the command prompt (or shell prompt on Mac/Linux).

Applications that run on desktop or laptop computers are called Desktop applications, while those that run on mobile devices are called Mobile apps. Whatsapp, Contacts, and Messaging are examples of mobile apps. A lot of applications these days can also be accessed by opening websites on the browser. Such applications are called "web applications". For example, GMail is a web application that you use by browsing to gmail.com.

Applications can be huge or they can be small. For example, an inventory management application might involve a large number of views with various options while a calculator application might show just one view. An application such as Volume Manager may even run as long as the computer remains switched on, completely in the background without showing any window to the user or it may run for a moment upon the user's request, do its job, and then terminate. It all depends on what an application is meant to achieve.

From a programmer's perspective, all applications are just programs written in one programming language or another.

1.3 Application Programming Interfaces

In the previous section I explained how programs are usually split into multiple parts so that some of the parts can be reused in other programs. The parts themselves may not necessarily look like applications with which humans can interact directly but they are programs nonetheless and they implement some functionality that can be used by other programs. For example, a program that interacts with the printer can be used by any application that requires something to be printed. If you ever attached a printer to your computer, you must have seen your computer trying to find a "driver" that can talk to the printer. This "driver" is nothing but an independent program that knows how to interact with that printer. Once the driver is installed, any application on that computer is able to use that printer automatically. How does that work? How is an application that knows nothing about a new printer able to use that printer to print?

Well, when you try to ship a package through the postal service do you know or care how exactly does the postal company deliver the letter to the destination? You simply wrap your package, put on the destination address label, fill out the required forms, and give it to the company representative at the counter. The rest is taken care of by the company. Basically, the company provides you with a simple "interface" to interact with them. Thus, shipping is a simple to use service but there are thousands of big and small mail order businesses that use the shipping service.

The same thing happens in software. A program that provides a service exposes an interface for other programs to use that service. This interface is called an **Application Programming Interface** (API) . If you are trying to develop an application that requires printing, your application would use the API provided by the driver to interact with the printer.

Just like a mail order business that uses third party companies for advertising, packaging, and shipping its products, you would focus on implementing the business logic of what you actually want your application do to and try to use third party services for tasks that are not unique to your application. Instead of writing everything from scratch, you would use the software libraries provided by someone else for such tasks. Some common services that applications usually require are interacting with files, interacting with the screen to show the user interface, interacting with mouse/keyboard, and communicating with other websites. All these services are implemented by someone other than the application developer. The application simply makes use of the APIs provided by the service providers to use their services. You may wonder at this point how exactly is an application developer supposed to know about the API provided by a software library. Simple. Documentation! If a library provider wants other people to use their library, they have to provide some kind of documentation explaining how exactly other people can use their library.

The reason I am talking about APIs is that a large part of gaining expertise in Java programming is gaining expertise in using the APIs provided by the Java platform. As you will soon see, Java comes with a huge library of reusable components, which makes developing Java applications easy as well as quick. But to harness the power of Java libraries, you must be mentally prepared to read a lot of documentation.



2. What is Java?

Exam Objectives

1. Describe the features of Java
2. Describe the real-world applications of Java

2.1 Understanding Java

You may have heard the story of the five blind men who tried to describe an elephant after observing it by touching. Since none of them had a complete view of the elephant, all of them described the elephant differently based on the part they touched. None of them were entirely incorrect in explaining what they observed but their understanding of the elephant was entirely incorrect. Thus, they deemed the elephant to be a completely useless animal. The situation with Java is similar. Java is not just one thing but a collection of multiple things. The word Java is also often used to mean different things depending on the context. Unless you understand the big picture, it will be difficult for you to take advantage of all the features that it provides. So, let me first explain all the different things that Java means.

Programming Language - First and foremost, Java is a programming language. Like all programming languages, it has its own set of keywords, syntax, and rules.

Java Development Kit - As explained before, code written in a programming language needs to be translated into instructions that can be understood by the computer. From that perspective, Java may refer to the tools such as compiler, debugger, and disassembler that help you write Java code. Collectively, they are called the Java Development Kit (JDK).

Java Virtual Machine - You may have heard of and seen different types of computers with different operating systems. You may be using a windows laptop, while your friend might be using an Ubuntu desktop. Your smartphone running Android or iOS is also a computer. All these machines have different architectures, different operating systems, and different features. You can't just copy an executable program such as Word from your windows machine to your MacBook and expect it to run. If you want to execute a program on any of these machines, you have to first translate the source code of that program into the machine language understood by that particular type of computer. Thus, you will need multiple translators to run your code on multiple machines and even then, due to differences in features, the resulting machine code may not work exactly the same on all the machines.

This is where the Java Virtual Machine (JVM) comes into picture. JVM is also a computer. Just like a regular computer, it has its own devices and features and it understands its own machine language. But unlike a regular computer, it is not a physical computer. In other words, it has all the virtues of a real computer but it does not exist physically, which is why it is called a "virtual" machine. A JVM is, in fact, a software program that mimics a computer and runs on top of a physical computer. Java designers identified the common features of regular physical computers and created a software program that simulates all those features. They translated this program into machine languages of all commonly used physical computers so that it can run on all those physical computers. Internally, the JVM uses the features of the physical computer on which it executes. Different types of computers require different JVMs but all such JVMs provide a single standard uniform interface.

So now, instead of worrying about different types of computers and instead of translating

your program into multiple machine languages for different computers, you just have to translate your Java code into the machine language understood by the JVM. You need to understand that instead of the computer executing your machine code directly, it will be the JVM that will be executing the machine code now. Furthermore, since a JVM is specific to a particular type of computer, it knows how to translate those instructions into the machine language understood by that particular type of computer.

Although Java designers have developed JVMs for all commonly used computers, there are still some types of computers for which there is no JVM. Most notably, there is no JVM available for Android and iOS based computers. The reason is that Android and iOS based systems do not have much in common with regular desktop computers. Therefore, it is not possible to run Java programs written for desktop computers on Android/iOS based smartphones. But the way technology is progressing for smartphones, it is entirely possible that there might be a JVM for these devices as well in future.

Java bytecode - The machine language understood by the JVM is called Java bytecode. If you want to run your program on the JVM, you must first "compile" that program into instructions written in Java byte code using the Java compiler provided by the Java Development Kit. The JVM interprets these bytecode instructions and further translates them into the machine code understood by the physical computer. Finally, the JVM uses the features provided by the Operating System to execute the machine code instructions. The reason why this machine language code is called "byte" code has to do with a bit of computer science theory, which is not important at this stage.

Java Class Library - As I mentioned before, new applications can be developed quickly if you reuse existing code instead of writing everything from scratch. Java comes with a huge library of code for common tasks that are required by Java applications. ranging from File manipulation to Graphical User Interfaces (GUI), from Networking to Databases, and from Security to Multi-threading. This library is called the Java Application Programming Interfaces (Java APIs). All the code in this standard library is organized in various "packages"package. For example, you can actually develop a complete database driven GUI application in a few lines of Java code by using the Swing package. The total size of your code turns out to be very small only because a large amount of code that already exists, is reused.

Java Runtime Environment - The Java Virtual Machine and the pre-compiled collection of the Java class library is called the Java Runtime Environment (JRE). Most desktop computers come with the a preinstalled but if a computer does not have it, one can easily download and install it for free. If a JRE is available on a particular machine, you can rest assured that your application will work the same on that machine without any change. Actually, there is a term for this - **WORA** - Write Once Run Anywhere! The JRE makes WORA possible. This is a big deal because before Java, programmers had to maintain different versions of their programs for different users depending on what type of computers they were using. They also had to test their programs on all these different platforms because there was no guarantee that even after compiling their program for a particular computer, it will behave the same. In practice, however, due to fundamental differences between different types of computers, a Java program might run a little

differently on them. Java GUI applications, in particular, do not look exactly the same way on Windows, Mac, and Linux computers.

It is important to understand that all of the above aspects of Java are independent of each other. For example, the Java language is used to develop Android applications even though there is no JVM or the JRE for Android. This is possible because Android uses the programming language aspect of Java and provides its own tools and libraries to translate Java code into the machine code that Android understands. The Java Virtual Machine, on the other hand has no dependency on the Java programming language. It is possible to write programs in several other languages such as Groovy, Jython, Kotlin, and Scala, and compile them into Java byte code. Of course, there will be different compiler programs for compiling code written in other languages, but once the code is compiled into bytecode, it does not matter which language was used to write the source code. Furthermore, some low powered consumer devices contain a JVM but do not contain the JRE. Since all these parts of Java work independently, it is possible to use which ever piece is suitable for a given purpose. This is one of the reasons why Java has been so successful.

The following figures illustrates the relative position of a JVM among various components of a computer.

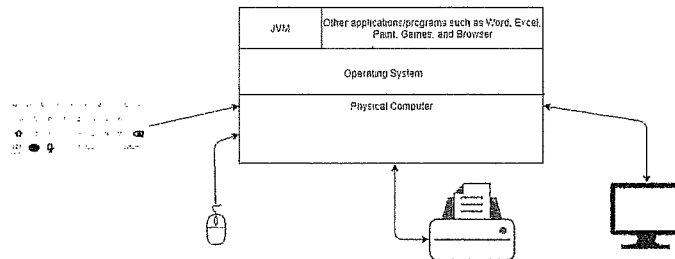


Figure 2.1.1: Relative position of the JVM, the OS, and the physical computer

In the above figure, observe that the JVM is just like any other application program running on a computer. Like all application programs, it takes help from the Operating System to use the resources of the physical computer.

The following figure explains the relation between JDK, JRE, and JVM.

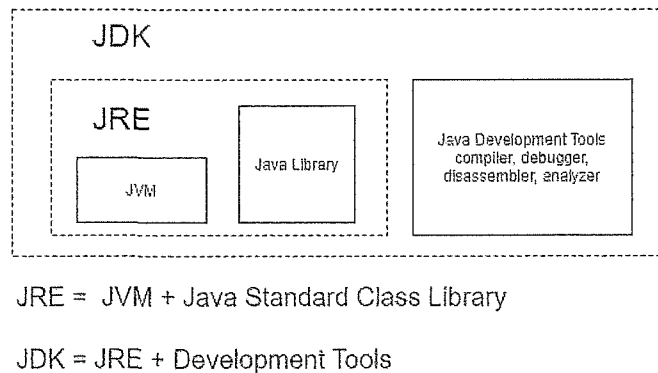


Figure 2.1.2: Components of a JDK and a JRE

Observe that the JVM and the Java standard class library (Java API) are collectively known as the JRE. Executing a Java program on a machine requires JRE. While developing Java programs on a machine requires development tools such as the Java compiler, which is provided by the JDK.

While the JDK includes basic development tools, many third party organizations provide even more advanced tools such as Integrated Development Environments (IDEs), dependency analyzers, and UML designers. These tools make software development in Java easier by highlighting keywords, syntax errors, and many such features. Commonly used IDEs are Apache Netbeans, Idea IntelliJ, and Eclipse. Of course, they are all optional.

Java Platforms - A Java Platform refers to a combination of JVM, Runtime Environment, and Development tools. Java has grown a lot over the years and it now contains so many features that the Java designers felt it necessary to create different packages for different needs. So, for basic needs, you have the **Java Standard Edition (Java SE)**. This package includes the JVM, Standard Class Library, and some development tools. This package is sufficient for developing standalone Java applications. For large enterprise applications, you have **Java Enterprise Edition (Java EE or JEE)**, which contains a server JVM, the Standard Class library, the Enterprise class library, and development tools. At one time, there was a package for mobiles and other electronic devices as well. It was called **Java Micro Edition (JME)**! All of these are Java platforms.

A Java platform is standardized for all devices. So, for example, if a machine or a device supports Java SE, then you can be assured that it supports all the components of the Java SE platform. Thus, if you develop an application on the Java SE platform, it will work the same on all machines that support the Java SE platform.

An important thing to mention here is that Oracle keeps tweaking the nomenclature and the actual contents of a Java platform as per the needs of the industry. For example, Java EE standard is now managed by a separate organization called Jakarta. So, Java EE is now called **Jakarta EE**!

Java versions - Java 1.0 was released in 1996 and as of this writing (Sep 2020), version 15 is the most recent version of Java. In between, there have been regular updates to all the

aspects of Java. Java 8, on which the JFC-JA 8 Certification (1Z0-811) is based, was released in 2014. Java 8 was a monumental version because a lot of new features and path breaking changes were done in this version. A majority of companies still run their software on Java 8. I will also be focusing on Java 8 in this book but I will also mention important changes that have happened to the Java platform after Java 8 occasionally.

Java Community Process - Early on (in 1998), an independent organization of various Java users (companies as well as developers) was formed to develop standard technical specifications for various Java technologies. This organization is named Java Community Process (JCP). JCP ensures that new features and changes are made in an organized fashion. It prevents Java from fragmentation into different flavors. For example, you will not have a Microsoft Java and an Oracle Java. There is always just one flavor of Java. Any one can implement their own Java compiler, JVM, and any other Java tool but they all must adhere to the standards approved by the JCP.

If all this terminology sounds too confusing, don't worry. It is confusing. Even for senior developers. You will get the hang of it in due course of time.

2.2 Features of Java

Now that you know various components of Java, it will be easier for you to understand the features of Java. In fact, several features are a direct consequence of the Java platform architecture itself. For this reason, I have categorized the features according to different perspectives. Let us check them out one by one.

From Management Perspective

1. **Platform Independent** - Java applications can be developed on one machine and run on another without the need to recompile the code.
2. **High Performance** - There was a time when Java was critiqued for its sluggish performance as compared to applications developed in C/C++. That is not so, anymore. Due to advancements in JVM, Java programs can run as fast or even faster than programs written in other languages.
3. **Secure** - Java has security features built-in. It is possible to run Java applications in a sandbox with access to only those parts of the host machine that are deemed necessary for that application, while everything else remains inaccessible. This is made possible by the Java Security Manager.
4. **Familiar** - Java belongs to the C/C++ family of languages and heavily borrows syntax and features from C/C++. This allowed existing C/C++ developers to become productive on Java quickly and paved Java's way to success.
5. **Simpler** - Java eliminated several complicated features of C/C++ such as pointer arithmetic, operator overloading, multiple class inheritance, that made applications error prone and hard to debug.

6. **Multiple delivery modes and deployment options** - Java applications can be delivered remotely to client machines in multitude of ways. For example, a rich GUI based Java application developed in Java Swing can be deployed on client machines over the internet using Java Web Start technology or a web based application developed using JSP/Servlet/Facelets can be delivered to the clients through a web browser.
7. **Java ecosystem** - Java programming community has developed tons of commercial as well as open source libraries for a variety of higher level tasks such as mail, encryption, PDF, and what not. A Java programmer is advised to first look for an existing class or library that performs the required task instead of writing one from scratch. This promotes rapid application development.
8. **Backward Compatibility** - There are many reasons for Java's success but if I were to pick one reason that has kept Java going so, strong even after 25 years, then backward compatibility would be it. An application developed on any old version of Java runs fine, or better in most cases, on newer JVMs without requiring any recompilation. Every new version of Java brought in performance enhancements and new features without breaking existing code. Two Java versions are most notable in this respect. Addition of Generics in Java 5 and Lambda Expressions in Java 8 were huge enhancements to the language but even these additions did not break existing code. This is not a trivial thing and this did not happen by chance. Java designers put in great efforts to make sure that existing applications get all the benefits of newer JVMs without requiring any change. Many new programmers may not appreciate how important this is but I can tell you that it is. Backward compatibility assures upper management that their investment in the given technology is protected. This makes the decision to use Java for developing new applications a no brainer.

From Technology Perspective

1. **Compiled** - Java code is compiled into bytecode using a compiler. The bytecode is then interpreted by the JVM. This makes Java a compiled language. Java bytecode is, of course, a low level language and is interpreted. Unlike scripting languages such as JavaScript or PHP, which are not compiled and are interpreted directly by JavaScript or PHP interpreter, compiled languages deliver better runtime performance.
2. **Variety of technological solutions under one umbrella** - The Java platform includes a variety of technologies for varying needs. For example, for rich client side application development (also known as Desktop applications), it provides Java AWT, Swing, and JavaFX. For developing web applications, Java has Java Server Pages (JSP). For server side applications, it provides Servlet, Enterprise Javabeans (EJB), Java Persistence Architecture (JPA), and Web Services. For communication between applications over the network, Java has support for raw sockets as well as higher level API such as RMI and XML-RPC. The technologies that I have mentioned here merely scratch the surface of the Java world. There are several more technologies, which you will learn about once you get into professional Java development.
3. **Multithreaded** - A multi-threaded program is able to manage multiple tasks at the same time. Java supports multi-threading and has features that make it easier to develop multi-threaded applications.

4. **Distributed** - Distributed applications are able to take advantage of multiple machines by distributing tasks to multiple machines. You might have one or more user facing GUI based clients running on one set of machines and a server that manages data centrally on another. Java has features that allows development of such applications.
5. **Garbage Collection** - JVM reclaims unused portions of program memory by removing unused objects. This process is called Garbage Collection. JVM provides various options for customizing garbage collection.

From Programming Perspective

1. **Object-Oriented** - Java is an Object-Oriented language. We will get into the details of this later.
2. **Structured** - Java belongs to the family of languages that follow the Structured Programming method. Structured programming involves modularizing large programs into smaller multiple pieces. This increases ease of maintenance and promotes reuse. Most commonly used programming languages such as C/C++, Java, C#, Python, and PHP, belong to this category. Other programming paradigms are Procedural and Functional. Practically, however, modern programming languages such as Java routinely borrow features of other programming paradigms. For example, Java has incorporated lambda expressions from functional programming.
3. **Statically typed** - Java is a statically typed language, which means, a variable and its type must be declared at compile time. Some languages such as JavaScript and Groovy, are dynamically typed. A dynamically typed language determines the type of a variable based on the value that it contains at runtime.
4. **Strongly typed** - Java is a strongly typed language, which means that after a programmer specifies what kind object or data a variable can point to, the variable cannot point to object of any other kind. For example, if you specify that a variable named car can point to objects of type Car, you cannot make that variable point to an object of type Tree in the middle of your program! This is very different from dynamically typed languages such as JavaScript. There are advantages and disadvantages of both the approaches but a discussion on that is beyond the scope of this book.
5. **Automatic memory management** - A Java programmer cannot manage program memory programmatically. In other words, a Java programmer does not directly write code to allocate or to release memory. A program simply creates objects as and when it needs. It does not even "delete" or "remove" objects explicitly. Allocation of memory for new objects and reclamation of unused objects is handled by the JVM completely automatically.
6. **Programmatic Exception Handling** - Java makes it easier to manage exceptional situations programmatically. As you will learn later, Java has try-catch constructs that helps you control program flow in case of unforeseen circumstances. It also provides constructs for closing resources automatically after use.

7. **Ready-made class library** - Java has a huge standard class library (Java API) that contains code for all sorts of common (such as File I/O and Database I/O) as well as uncommon (such as cryptography) tasks.

Don't worry if you don't completely understand every one of these immediately. You will gain better understanding as you go through the book.

2.3 Real world Applications of Java

Java was originally designed with intelligent electronic consumer devices in mind. The idea was to create a uniform platform for controlling various kinds of consumer devices that have limited memory and processing power. Although Java has been successful in that domain, it has been found even more useful in developing high performance distributed enterprise applications. Many kinds of applications have been developed in Java over the years and it would not be possible to list all of them here. Broadly, however, Java applications can be classified as follows:

Desktop Applications

Java has been used to develop standalone desktop applications with or without a Graphical User Interface (GUI). A non-GUI application can be run on a machine as a service or launched from the command line as and when needed. For example, you can have a screen capturing application running silently in the background. Wide range of GUI based applications such as simple contact management to complicated ERP applications have been developed using Java's AWT and Swing package. Enthware's popular mock exam simulator is a good example of a standalone GUI application of medium complexity.

In early years of Java, one more type of GUI based Java applications called "Applets" were popular. Applets were small applications that were loaded in a browser window when the user opened a website containing them. They soon ran out of favor due to sluggish performance and security restrictions. Applets have been deprecated in Java 8.

Distributed Applications

Many applications provide some sort of service to the users and to provide that service lot of activities are performed at a single location and the results are displayed to several clients. For example, an airline ticket reservation system manages the tickets and reservation data in the company, but the users view and book the tickets on their computer using a GUI. Thus, this is a distributed application where there is a client component that communicates with the server component. Various pieces of an application can communicate with each other using Java RMI.

Web Applications

These days most enterprise applications such as reservation systems, banking operations, shopping sites, are accessed by the users from the browser. Users access them just like regular websites without having to install anything on their machines. Java Server Pages (JSP) was a popular choice for building the user accessible parts of such application. In recent times, however, the user interface is built using JavaScript but the server side components are still built using Java Servlet,

EJB, JPA, Web Services, and many other Java technologies.

Middleware Applications

Establishing communication between two components of a distributed application is a non-trivial task in itself. There are different approaches to manage such communication. Applications such as message brokers and protocol handlers are meant to do exactly that. They sit in the middle of two applications and help them communicate. Several applications such as Tomcat Servlet engine, JBoss Application Server, Tibco messaging service are developed in Java.

Server side Applications

Java has been used to develop purely server side application such as Database engines or data analytics platforms. It has been used to develop web servers and file servers that implement the HTTP/HTTPS and FTP protocols.

Frameworks and Libraries

A huge number of open source frameworks and libraries have been developed by Java developers. Many of these frameworks such as Struts, Spring, and Hibernate form the back bone of several enterprise applications.

Mobile Applications

Before the advent of Android and iOS, Java was used to develop applications for mobile phones using Java Micro Edition platform. Although this is now completely outdated, Java as a programming language is still used to develop applications on Android.

Java has been used to develop applications in several domains. Some of the interesting ones are Scientific applications including research and engineering, Business applications such as Tax filing, Stock trading, Banking, and ERP, Media applications such as animations and games, Consumer applications such as browser, email clients, and media players, Data analysis applications such as Big Data, Data Mining.

The list of Java applications is huge and if you google the uses of Java, you will be truly astonished to see what all people have developed using Java. In this book, you will learn to make simple Java applications that can be run from the command line. But the rules of the language that you will learn in the process will enable you to make much more advanced Java applications in the future.

2.4 What Java is not!

Nothing except the almighty is perfect and one solution never fits all. There are certain things that you can't do in Java. Some because of imperfection and some because it was never designed for them. Here are a few things that you should know in this respect.

1. **Not an executable:** The exe files that you see on Windows machines are called native executables because the operating system (Windows) can understand and execute the instructions contained in them directly. Java programs can't be run like those executable files.

When you compile Java code, you don't get an "exe" file that you can just double click and run. The result of compiling Java code produces "class" files, which are interpreted by the JVM (and not by the operating system). The JVM is the executable program here. A class file is merely the input that you give to the JVM.

Over the years, however, several approaches to "execute" Java programs have been developed. Most involve some kind of a launcher application that launches the JVM with appropriate command line options (command line options are also called "switches", by the way). This launcher application could be an exe or a simple batch file.

Another approach is to "associate" class files with "java.exe". For example, when you double click on a text file, Windows launches Notepad application with the file that you double clicked on as a parameter. Or when you double click on an image file such as jpg or png, Windows launches the MS Paint or the Photos application. Similarly, compiled Java code files can be associated with the JVM program. I will show you both the approaches later.

2. **No code level security** - Unlike native executables, compiled Java code can be easily decompiled to produce Java source code that is almost the same as the original source code that was written to generate the compiled code. When I was a newbie Java programmer, I wrote a cool Java program. Then I thought what if someone just reverse engineered it. I tried to find ways to create non-human readable exe files of my program. I soon found out that it is not always possible to do so. There are tools that can convert class files to native exes but there are so many limitations to this process that it is not worth doing. There are tools that "obfuscate" Java code, which basically make the source code very hard to read. Thus, even if someone decompiles the class files, the result will still not be understandable. The point to take away here is that it is easy to replace a class file with a hacked one.
3. **Clunky GUI** - The Graphical User Interfaces built in Java do not compare favorably to the GUIs built using native operative system libraries. Not only that they don't look and feel as good as the native GUIs, they don't work the same on all platforms. The GUI framework of Java negates the WORA feature of Java. It makes Java a "write once test everywhere" language. For this reason, Java is used a lot more for developing server side components than for rich client application.
4. **Not on Mobiles** - As I mentioned before, there is no JRE and no JVM for Android and iOS devices as of now. So, even though smartphones have become quite powerful you still can't run Java applications on them.
5. **High learning curve** - This point is not entirely correct but comes up very often. Java designers actually designed Java to be a lot simpler than C/C++. However, by some perspective, several newer languages such as Python made things even simpler. It is definitely easier to start writing programs in Python. I personally believe that as soon as you move towards writing non-trivial commercial applications, you will hit the ceiling with such languages. The extra effort that you put in initially towards learning Java pays off handsomely later.
6. **Language shortcomings** - As they say, hindsight is always 20/20. There are certain things in the Java programming language that could have done better from the get go. For example, Java has been criticized for not having a decimal datatype or unsigned datatypes. There are

many such things but most of them are quite subjective and a detailed discussion on them is beyond the scope of this book.

7. **Can't please everyone** - Java addresses a wide range of requirements, yet, it is entirely possible that it is no good for some use cases. For example, you can't write device drivers in Java because, again, Java programs run on the JVM and not directly on the machine.



3. Java Basics

Exam Objectives

1. Describe the Java Development Kit (JDK) and the Java Runtime Environment (JRE)
2. Describe the components of object-oriented programming
3. Describe the components of a basic Java program
4. Compile and execute a Java program

3.1 Installing Java

In this section I will show you how to get all the tools on your machine and how to set up the environment on your machine to use those tools. But first, I need to introduce some basic commands that we will be using often.

3.1.1 System Check

The two most important things that you should be handy with are the **File Explorer** and the **Command Prompt**. Let's check them out.

File Explorer

Windows 10 has various ways to open the File Explorer. The easiest way is to press the Windows key and the E key at the same time (Win+E). The File Explorer window will show up.

By default, JREs and JDKs are installed under C:\Program Files\Java folder. The following screenshot shows all the JREs and the JDKs that are present on my machine. You might see a JRE in this folder as well but we are more interested in the JDK. So, if you see a `jdk1.8.*` folder, great! You already have the JDK version that we need. If you don't or if you have an older version of the JDK (such as `jdk1.7`), that is fine too because I will show you how to install the most recent JDK shortly.

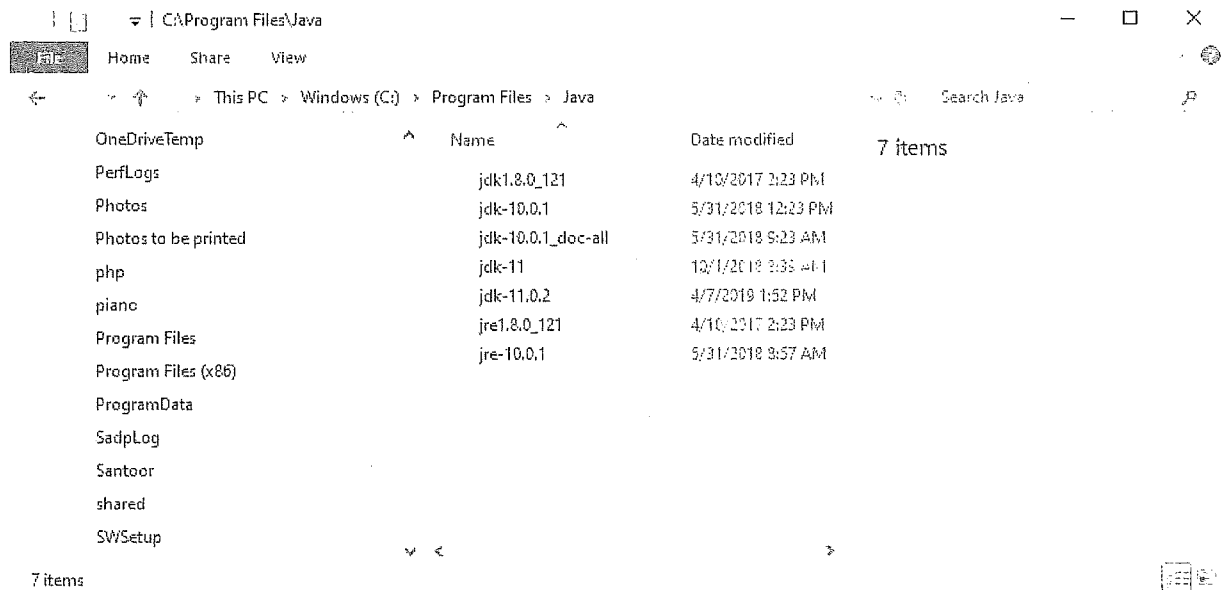


Figure 3.1.1: Looking for previous Java installations

Command Prompt

Press Windows+R keys to open the "Run" box. Type `cmd` in the run box and then click "Open" as shown in the following screenshot:

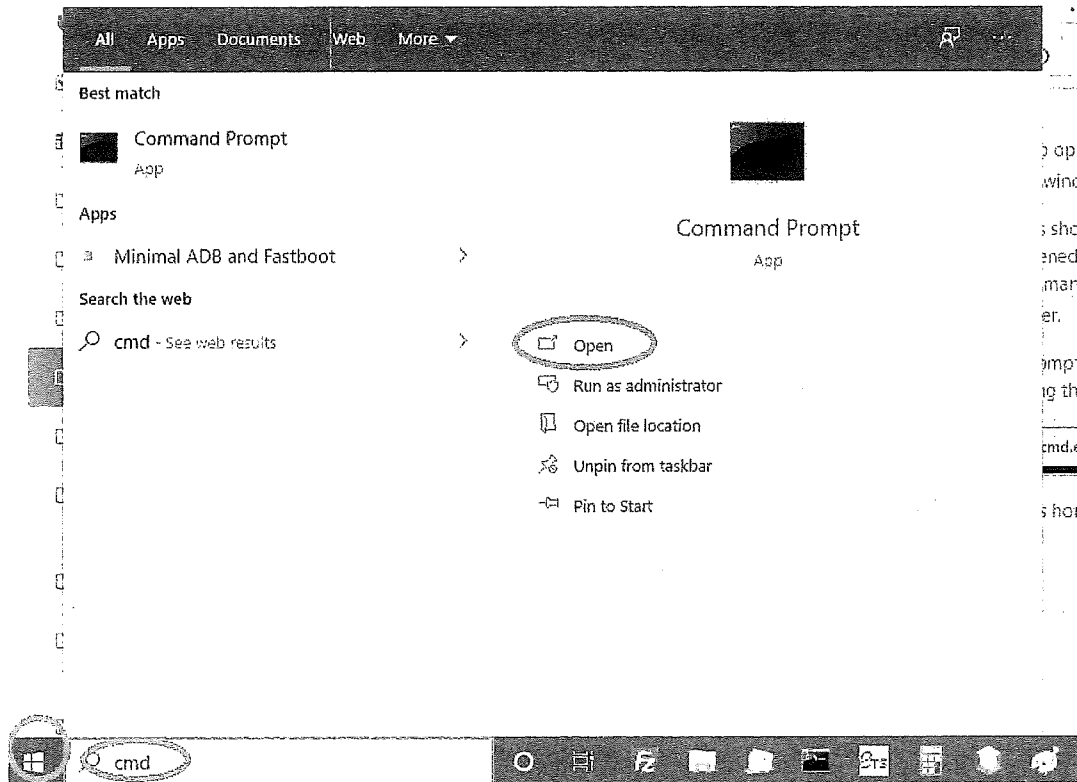


Figure 3.1.2: Opening Command Prompt/Window

You should see the following window after clicking on "Open".

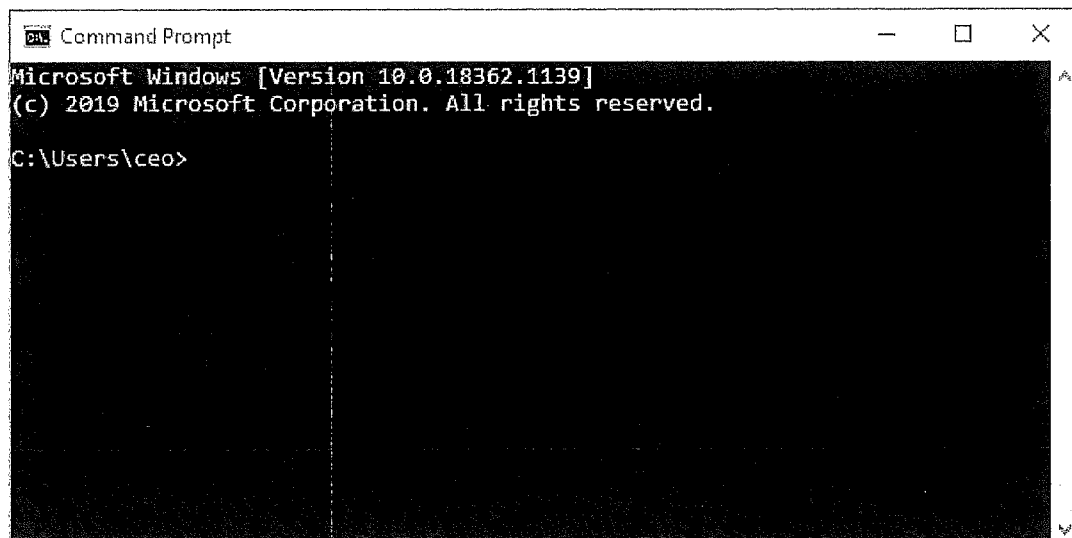
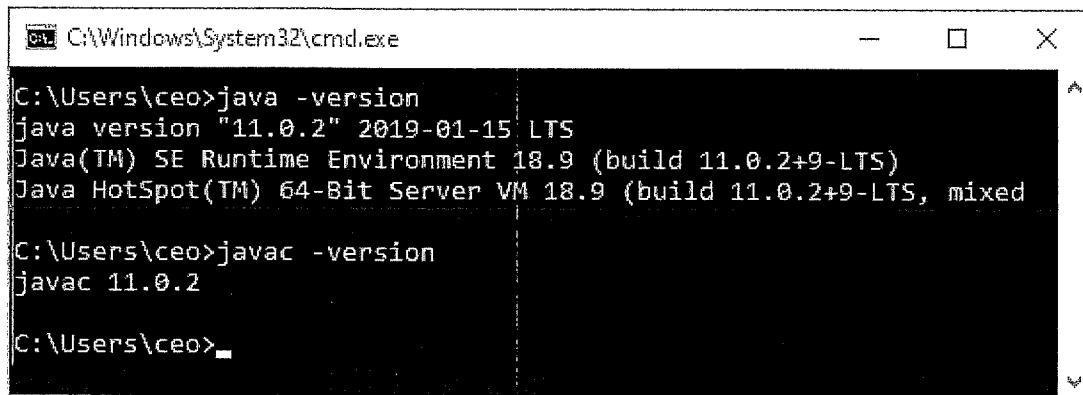


Figure 3.1.3: Command Prompt/Window

Observe the folder that is shown on the command window. In the above screenshot, it says `C:\Users\ceo`. This is my home folder. You should also browse to this folder from the File Explorer that you opened above. Create a new folder under this folder with name **jfcja**. You

can create it from the File Explorer or from the command prompt using the `mkdir` command (`C:\Users\mike> mkdir jfcja`). We will keep our test programs under this folder.

While your command prompt is open, let us also check whether you already have a JRE or a JDK configured on your machine. You can do this by running the `java` and `javac` commands on the command prompt as shown below.

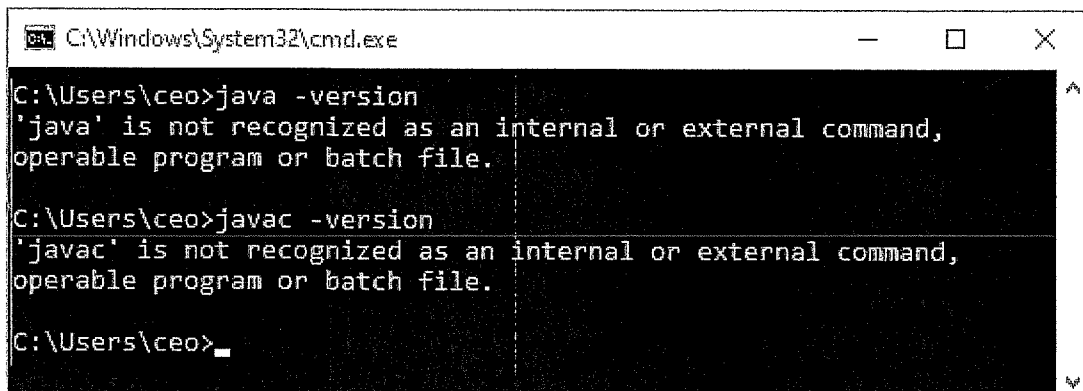
A screenshot of a Windows command prompt window titled "C:\Windows\System32\cmd.exe". The window has a black background with white text. The user has entered two commands: `java -version` and `javac -version`. The output for `java -version` is: `java version "11.0.2" 2019-01-15 LTS`, `Java(TM) SE Runtime Environment 18.9 (build 11.0.2+9-LTS)`, and `Java HotSpot(TM) 64-Bit Server VM 18.9 (build 11.0.2+9-LTS, mixed`. The output for `javac -version` is: `javac 11.0.2`. The prompt is currently at `C:\Users\ceo>`.

```
C:\Windows\System32\cmd.exe
C:\Users\ceo>java -version
java version "11.0.2" 2019-01-15 LTS
Java(TM) SE Runtime Environment 18.9 (build 11.0.2+9-LTS)
Java HotSpot(TM) 64-Bit Server VM 18.9 (build 11.0.2+9-LTS, mixed
C:\Users\ceo>javac -version
javac 11.0.2
C:\Users\ceo>
```

Figure 3.1.4: Successful check for existing Java installation

The above image shows that I ran the `java` command with the `-version` option. It ran successfully and printed the version information. The output tells me that I have JRE version 11.0.2 installed and configured on my machine. Furthermore, the `javac` command also showed me the same version. This means, I have JDK version 11.0.2 configured on my machine. If you get a similar output that shows a Java version greater than or equal to 8, your machine has a JRE and a JDK installed and configured correctly. You are all set and you don't have to worry about the next section.

If your machine does not have any JRE and JDK configured, you will get the following output:

A screenshot of a Windows command prompt window titled "C:\Windows\System32\cmd.exe". The window has a black background with white text. The user has entered two commands: `java -version` and `javac -version`. The output for both commands is: `'java' is not recognized as an internal or external command, operable program or batch file.` and `'javac' is not recognized as an internal or external command, operable program or batch file.` respectively. The prompt is currently at `C:\Users\ceo>`.

```
C:\Windows\System32\cmd.exe
C:\Users\ceo>java -version
'java' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\ceo>javac -version
'javac' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\ceo>
```

Figure 3.1.5: Unsuccessful check for existing Java installation

The outputs of both the commands say the same thing. Windows was not able to find `java` and `javac` programs on your computer. It means that either your machine does not have a JRE and a JDK installed, or if it is installed, Windows does not know where it is. If the `java` command shows version information but the `javac` command doesn't, that means you have a JRE but no JDK. Either way, you will need to install the right JDK.

For Mac and Linux

I have not included the steps to open the File Explorer and Command window for Linux and Mac users in this book because I am not sure if there are that many readers who require such detailed instructions. If you need instructions for your OS, please let me know and I will post the instructions online.

3.1.2 Understanding Java Builds

Before you go about installing Java, you need to be aware of a very important change in Oracle's licensing agreement. Until Java 10, Oracle's JDK was free to download as well as to use for development, demonstration, testing, and production purposes. However, since Java 11, Oracle does not allow the use of their JDK in commercial environment for free. It is still free to be used for other purposes but if you want to use it in production (meaning, the environment that is used to run your business), you need to purchase a license from Oracle. Does that mean Java is no more free? No, that's not exactly true.

Oracle has created a group called OpenJDK and has released the source code for the JDK on openjdk.java.net. Companies take that same source code and create their own version of the Java and the JDK. Each such version is called a "Java Build". Every Java build must also get a certification from OpenJDK that it satisfies all the criteria for being called a Java Standard Edition (Java SE) build. Oracle's official Java build is one of those builds and it is that particular build that is not free for commercial purpose. But many other builds, including the OpenJDK build, is free even for commercial use. So, basically, Oracle has two Java Builds - the official Oracle Java, which is not free for commercial use, and OpenJDK Java, which is free for commercial use.

Since the source code for all the builds is the same, there are all practically the same. The difference is in how soon security patches and updates are applied to a build. The difference is also in the support and help that is provided by the builder to the users of their build.

Among the free ones, the build provided by OpenJDK is considered to be the most updated and is also the build of choice for most companies.

The reason I needed to mention the above facts is that while the JFCJA exam is based on Java 8, the latest version of Java as of May 2024 is 22! The question is, if you do not already have JDK 8 installed is it still worth spending time on installing the old JDK when you can work with the latest?

Several changes and additions have been done since Java 8 and normally, I recommend people to use the version of Java on which the exam is based. However, in this case, since this certification exam focuses on only the basics, most of the features added in recent Java versions are beyond the scope of the exam anyway. Therefore, I think it is alright if even you get the latest Java Build.

3.1.3 Downloading and Installing OpenJDK 15

Armed with the above knowledge, you can decide which Java would you like to have on your machine. I will go with the latest OpenJDK, which is available at jdk.java.net. On this site, you will see all the versions of the JDK that are available. I picked Java 15 and upon clicking on Java 15, the following is what I get.

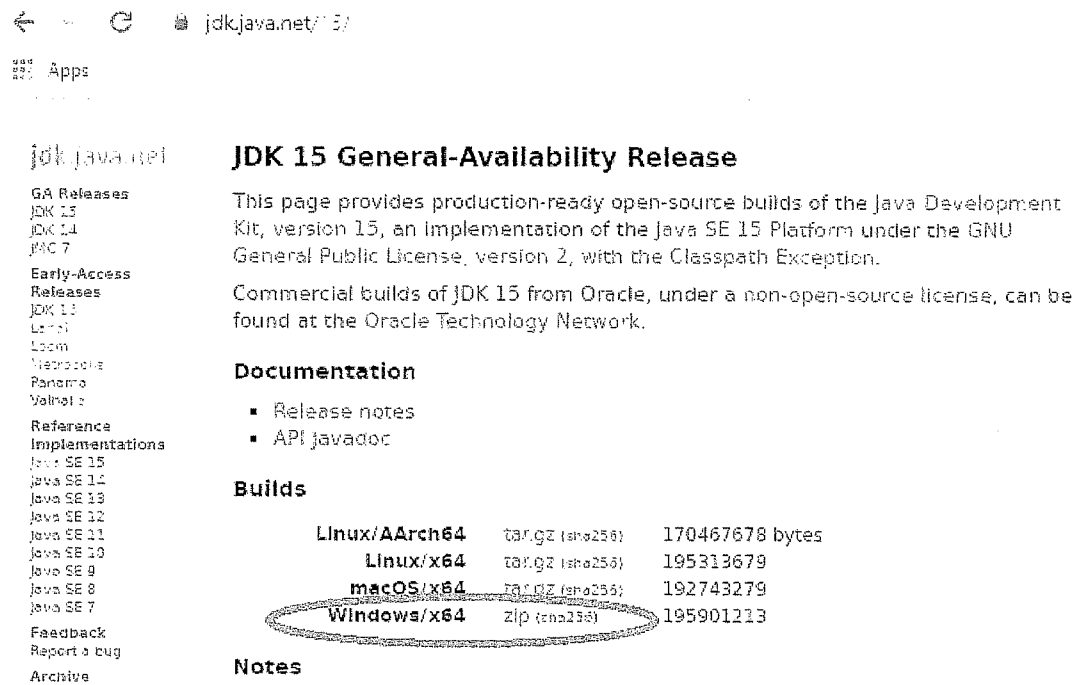


Figure 3.1.6: Downloading OpenJDK 15

You can see four different downloads. One for each of the four commonly used operating systems. I will show you the installation process for Windows first and then for MacOS and Linux.

Step 1. Download Zip

Click on the "zip" link (on the last row, starting with Windows) to download the `openjdk-15_windows-x64_bin.zip` file and save it to your default downloads location. For example, if your user name is mike, your default download location on Windows 10 is `C:\users\mike\Downloads`. If you have changed your default downloads location, that is fine. Just note down the location where you are saving the zip file.

Step 2. Extract Zip

Open the File Explorer and go to the location where you have downloaded the zip file. Right click on the downloaded zip file and select the "Extract all..." option. You should see the following dialog:

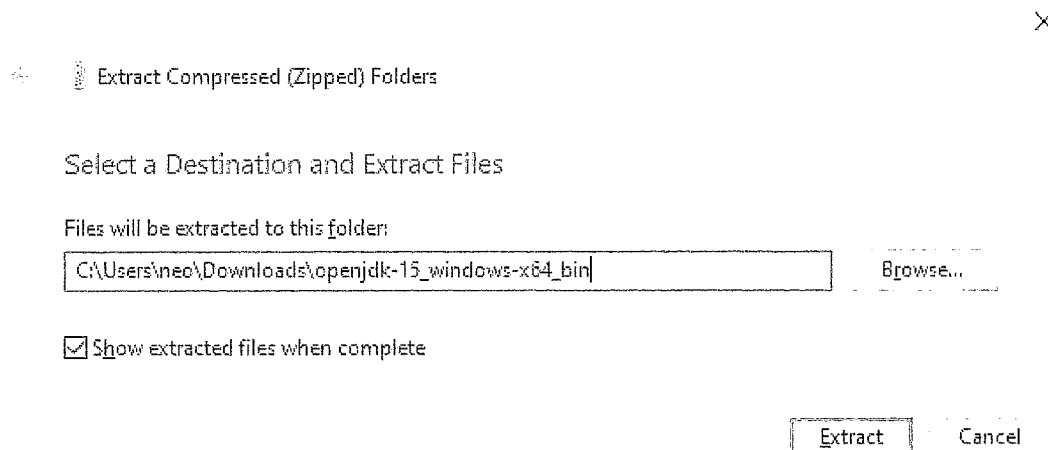


Figure 3.1.7: Extracting the contents of the jdk 15 zip file

You can extract the contents to any directory but I recommend extracting them to **C:** because that will make your life a bit easier. To do so, change the destination directory to **C:** before clicking Extract. You may be prompted for administrative permission to make changes to **C:**. If you do not have that and you cannot get help from the administrator of your machine, you may extract it to any other folder under **C:\Users\<your user name>** such as **C:\Users\mike**.

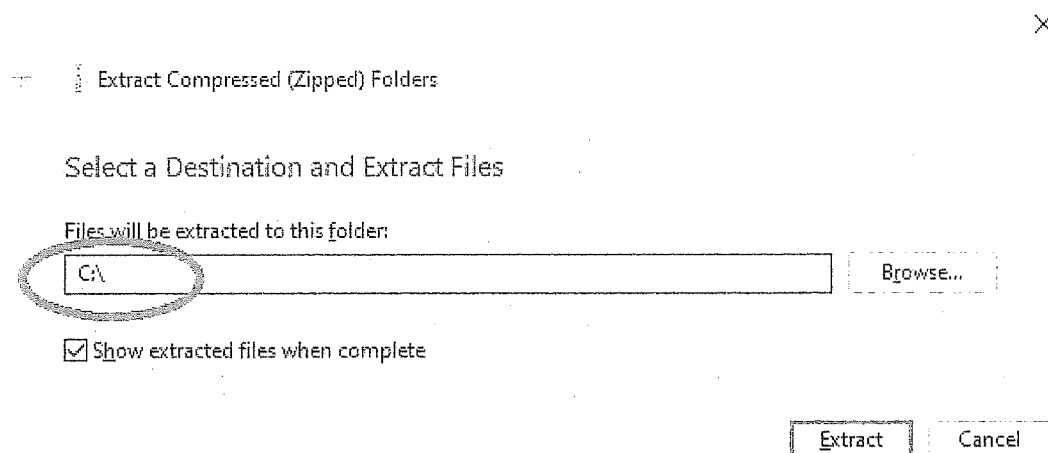


Figure 3.1.8: Extracting the contents to C:

Step 3. Verify the extract

After extracting the contents, you should have a new folder named **jdk-15** under **C:** (or under whichever directory you decided to extract the contents in). Under **jdk-15**, you will see several folders including **bin**.

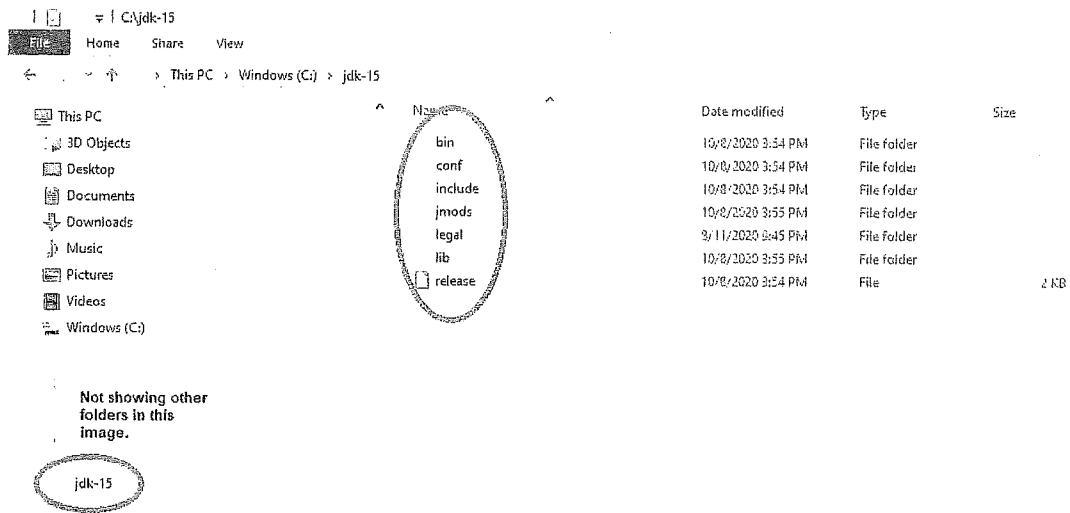


Figure 3.1.9: Verify the contents of jdk installation folder

If you double click on the bin folder, you will see several executable programs. The ones we are interested in right now are `javac.exe` and `java.exe`. `Javac.exe` is the Java compiler program. We will use it to compile our Java code. `Java.exe` is the JVM program. We will use it to execute our compiled Java code.

Step 4. Setting up the PATH environment variable

Technically, your Java 15 installation is done. You can open up a command prompt and run the following command:

```
c:\jdk-15\bin\java
```

You will see the output shown in the following image:

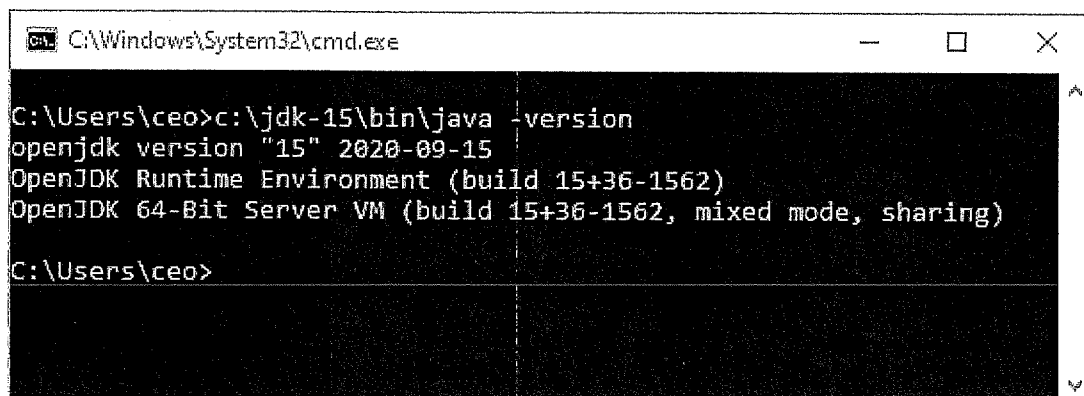
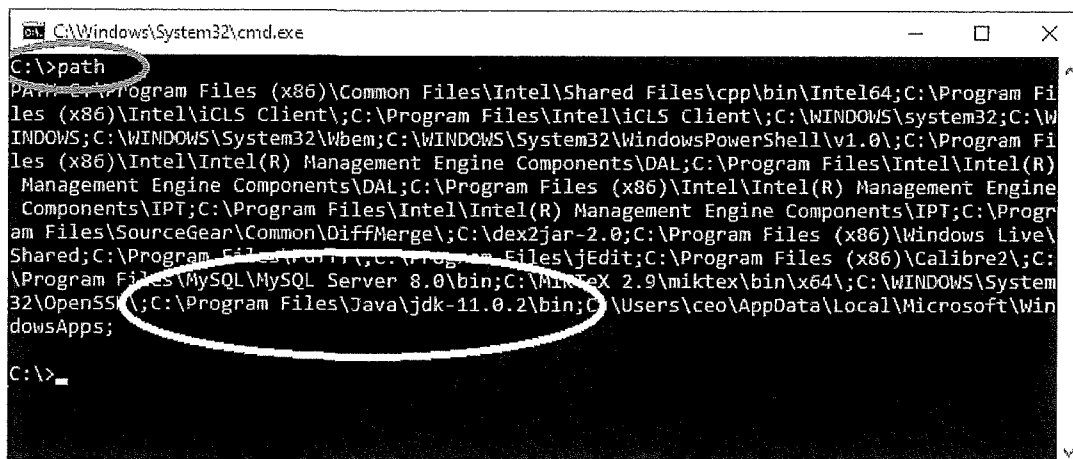


Figure 3.1.10: Check Java version after installation

In the command that you just executed, you specified the full path (`C:\jdk-15\bin`) to the `java.exe` executable because the OS has no idea that it needs to look in the `C:\jdk-15\bin` directory to find `java.exe`. You will need to type `C:\jdk-15\bin` every time you want to compile Java code and execute it. This is too tedious. To make the OS look into the `C:\jdk-15\bin` folder when you try to execute `java`, you need to add `C:\jdk-15\bin` folder to the environment variable named **PATH**. The **PATH** environment variable tells the operating system the locations where your

executable programs may be located. Once you add a location to PATH, the OS will automatically search that location for the executable file of the program whenever you request the OS to execute a program.

Before messing with PATH, it is a good idea to check what it already has by executing the path command on the command window. The following is what I see on my machine. You should see something similar.



```
C:\Windows\System32\cmd.exe
C:\>path
PATH=C:\Program Files (x86)\Common Files\Intel\Shared Files\cpp\bin\Intel64;C:\Program Files (x86)\Intel\iCLS Client\;C:\Program Files\Intel\iCLS Client\;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\WINDOWS\System32\WindowsPowerShell\v1.0\;C:\Program Files (x86)\Intel\Intel(R) Management Engine Components\DAL;C:\Program Files\Intel\Intel(R) Management Engine Components\DAL;C:\Program Files (x86)\Intel\Intel(R) Management Engine Components\IPT;C:\Program Files\Intel\Intel(R) Management Engine Components\IPT;C:\Program Files\SourceGear\Common\DiffMerge\;C:\dex2jar-2.0;C:\Program Files (x86)\Windows Live\Shared;C:\Program Files\jEdit;C:\Program Files (x86)\Calibre2\;C:\Program Files\MySQL\MySQL Server 8.0\bin;C:\MikTeX 2.9\miktex\bin\x64\;C:\WINDOWS\System32\OpenSSH\;C:\Program Files\Java\jdk-11.0.2\bin;C:\Users\ceo\AppData\Local\Microsoft\WindowsApps;
C:\>
```

Figure 3.1.11: Check existing value of PATH

The output lists all the folders present in the path. Whenever I try to execute any program on the command prompt, Windows will search for that program in all these locations one by one and execute it as soon as it finds the program. Windows will not look at the rest of the locations once it finds the program.

Observe that I have the bin folder of jdk-11.0.2 in the path. So, if I try to execute just java (instead of C:\jdk-15\bin\java), Windows will pick up the java.exe executable file from C:\program files\Java\jdk-11.0.2\bin. Since I am trying to work with JDK 15, I would like Windows to pick java.exe from C:\jdk-15. For that to happen, I need to adjust my PATH variable. Specifically, I need to remove the existing Java 11's bin folder from PATH and add the new Java 15's bin folder to it.

Click on Windows Start/Search and type env.

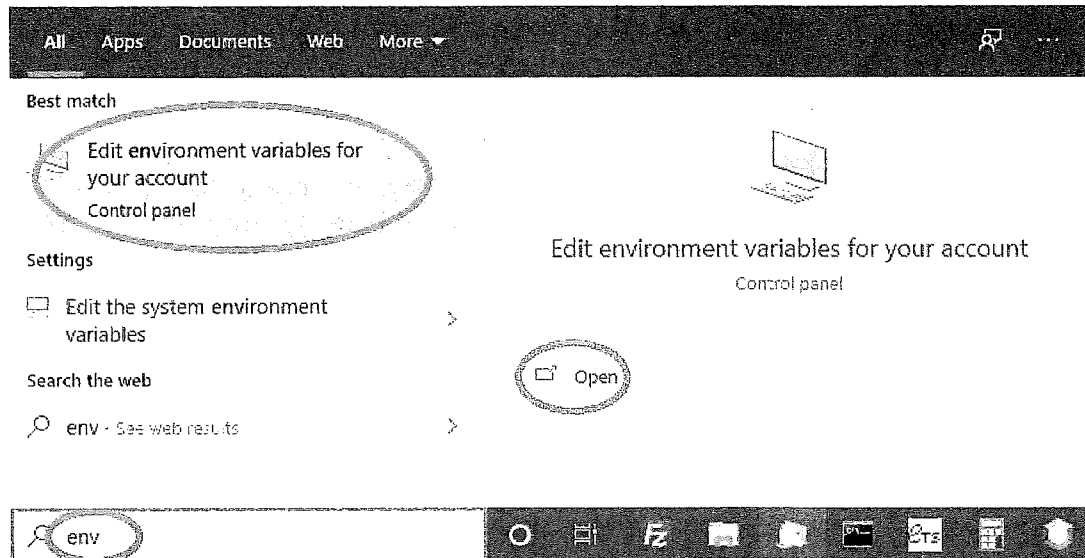
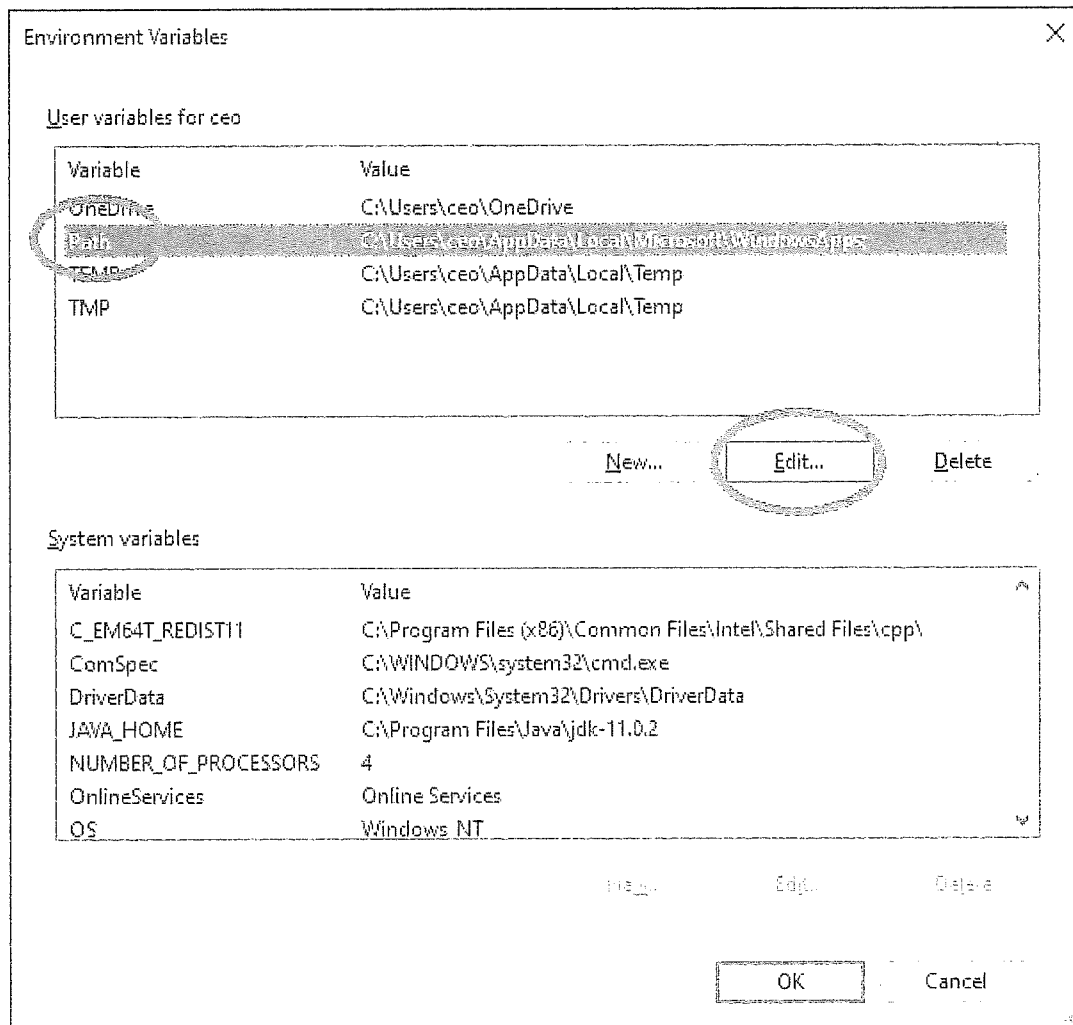


Figure 3.1.12: Edit environment variables

Observe that there are two different sets of environment variables - one for your own account and one for the system. The difference is that the variables defined for your account are visible only when you are logged in, while the variables defined in the system are visible to all the users. Thus, the actual value of an environment variable is the summation of both the values, with the system variables taking precedence over user account variables. If you did not see any JDK or JRE folder listed in the output of the path command, you can edit your own account's PATH variable. Otherwise, you need to check where exactly the path to the jdk or the jre is present and edit that PATH variable. Editing System's variable will require admin permissions but the process is the same. The following two screenshots show how to edit the existing PATH variable.

**Figure 3.1.13: Edit PATH**

After selecting the PATH variable and clicking on Edit, click on New to add a new folder to the path.

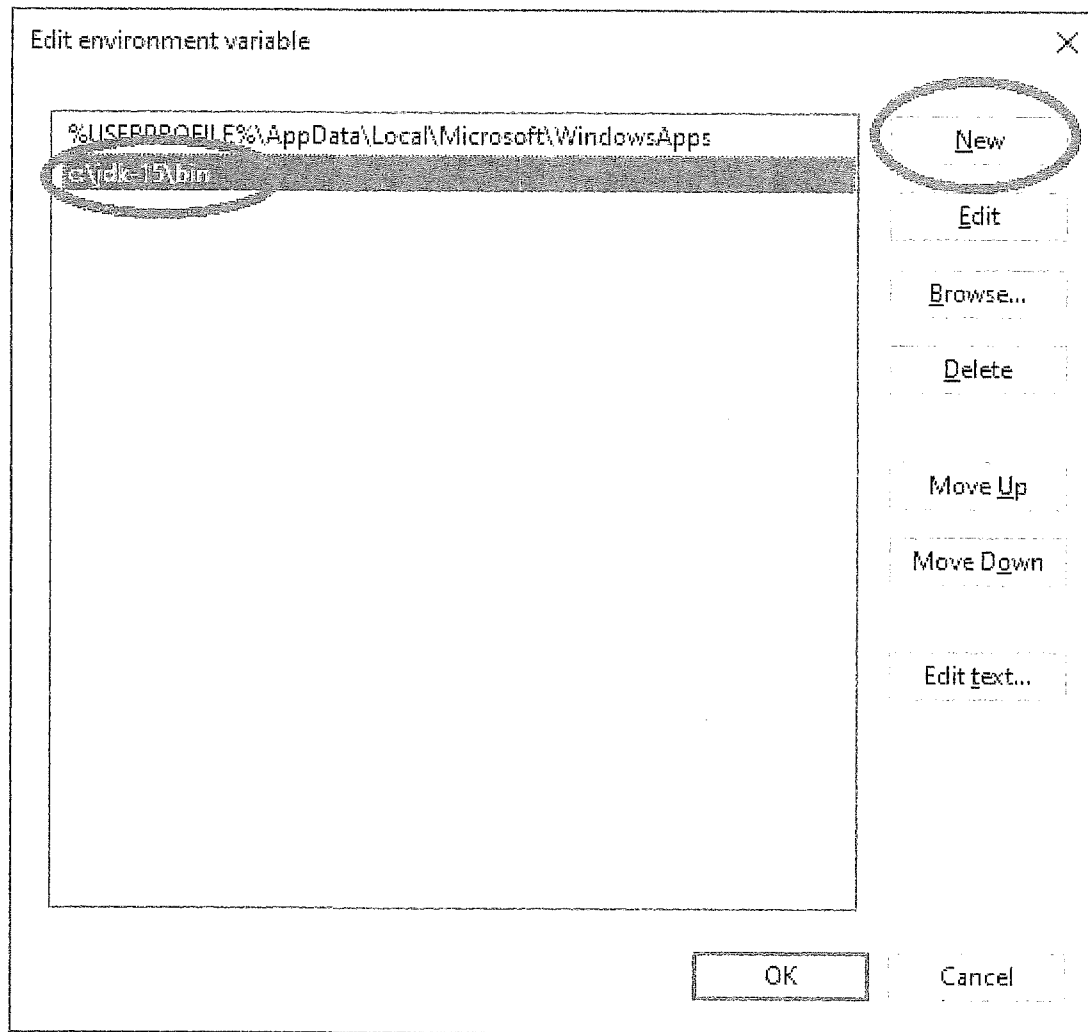


Figure 3.1.14: Add jdk's bin folder to PATH

Click OK one by one on all the open dialogs to save the changes. The change to the PATH variable will not be visible in the command windows that are already open. Close all of them, open up a new command prompt, and run the java and javac commands. You should see the following output:

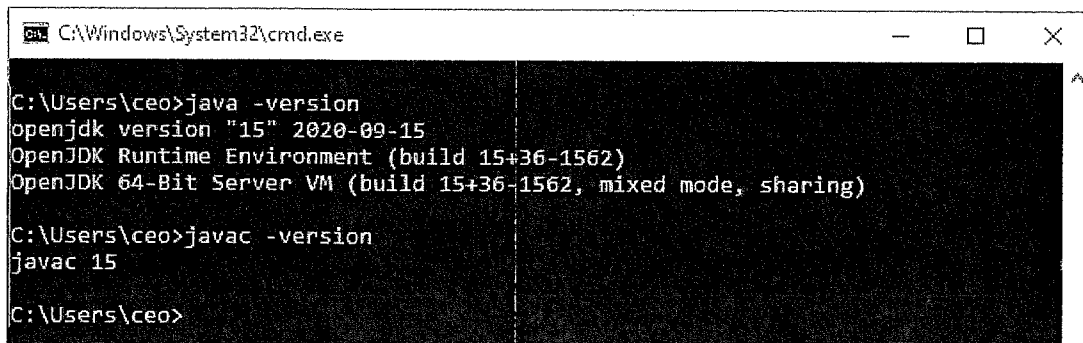


Figure 3.1.15: Run java after updating PATH

The above screen shot shows that you are now able to run the correct version of Java as well as javac. This means, you are now ready to write and execute some Java code!

3.1.4 Using an Integrated Development Environment (IDE)

Code is, after all, just plain text. Decades ago, professional programmers used to write code using simple text editors. These were general purpose text editors that knew how to handle text but had no idea what that text meant or what that text was meant for. If one made a typing error in their code, they would find out only when they tried to compile the code in a separate command line window. The compiler would generate an error message and the developer would then fix the code and compile it again.

Soon, as applications grew, the size of the code became too big to be managed in this manner. This encouraged developers to use special purpose text editors that understood the programming language text that was being written. They highlighted keywords, syntax errors, and typing errors in the code. This increased the efficiency of the developers by eliminating the need to manually compile the code to find out errors.

State of the art applications nowadays contain not just code but also resource files such as images, properties, and configuration information. A project may contain thousands of files organized in hundreds of directories. A small change in one part of the code may have a cascading effect on hundreds of other code files. More over, there are multiple developers working on different parts of an application. It is practically impossible for a developer to manually manage all the code for an application using a simple text editor and the file explorer. This is where an Integrated Development Environment (IDE) comes into picture.

An IDE adds a lot more programming language specific features on top of text editors that help developers write good quality code. Besides highlighting syntax errors, they generate code based on commonly used idioms, analyze impact of a change on other parts, and compile the code transparently. They also provide support for debugging the code by executing it step by step. It would not be an exaggeration to say that professional software development is practically impossible without a good IDE.

The Java world has several free as well as commercial IDEs. Top three are - **Eclipse** (free), **Netbeans** (free), and **IDEA** (commercial). These are professional IDEs and have lots of features. But they are quite heavyweight. Meaning, they require a lot of efforts to install and have a relatively higher learning curve. But once you spend some time in learning their ways, they save a ton of time later. There are several lightweight IDEs as well, which are meant for beginners. They do not take much to install and have a low learning curve but they do not provide as many features either. **Dr Java** and **BlueJ** are two free IDEs in this category.

I know what you might be thinking now. No, I am not prepping you for downloading and installing an IDE. Gaining expertise on an IDE is extremely important for your productivity but you cannot achieve this expertise without learning the fundamentals. Under the hood, an IDE does the same things that programmers used to do manually. It just does those things automatically and transparently. Even though you may not need to do those things manually, you must still know what exactly is the IDE doing. Therefore, it is highly recommended that one should learn the basics of the language without using an IDE and move to a professional IDE later.

Another problem with using an IDE at this stage is that IDEs are not too quick to upgrade to the latest version of Java. For example, as of this writing none of the IDEs support Java 15. I am

sure, they will start supporting it soon but they don't at this moment. This is not a concern for professional application development because applications and teams do not switch to a new Java version that quickly either. In fact, even today, six years after its release, Java 8 is still the most used Java version!

So, we will start writing our Java code with a basic text editor. I prefer **Notepad++** on Windows. Unfortunately, it is not available on Mac and Linux. If you are using MacOS, you can go with **Atom** and if you are using Linux, you can use **Notepadqq**. But again, since we will be writing simple and short programs, the editor doesn't matter. You can use whichever text editor you have.

3.2 JDK and JRE

I explained the relation between JDK and JRE in the previous chapter. To recap, a Java Runtime Environment includes just the stuff necessary to run a Java program. This means, the JVM, the Java standard classes (aka the Java API), and its configuration and extension files. A Java Development Kit includes tools that are required to write Java programs such as the Java compiler and the debugger. A JDK contains a JRE and is therefore, much larger in size than a JRE. If you have JDK 8 installed on your computer, you will see a JRE folder inside the JDK folder.

The purpose of treating JRE and JDK as two separate entities was to ensure that a runtime environment could be preinstalled on all computers. This allowed Java application developers to ship Java applications without worrying about installing Java on the user's machine first.

Another reason for the split was related to licensing. Java application developers were allowed to bundle a JRE with their application without paying any fees to Oracle. This took care of the cases where the user's machine did not have a JRE preinstalled or had a different version of JRE.

The situation has changed drastically with Java 11. Starting with Java 11, Oracle has stopped bundling JRE with JDK. In fact, there is no standard JRE anymore. There is just the JDK. So, if you look inside the Java 11 (or higher) installation folder, you will not find a JRE folder.

However, the JDK itself has been modularized and depending on what features of the JDK your application uses, you can pick just those parts of the JDK and bundle them with your application. Licensing fees will depend on which "build" of Java you are using. For example, as explained before, Java is free while Oracle Java is not.

Note that bundling and distributing Java with an application is not a trivial matter and the above details merely present a broad overview of the issues involved. However, it is not something that you should be concerned with at this stage.

3.3 Components of a Java program

Now that you have already how a Java program looks like, let's get into the components a Java program.

1. **The source code:** All the source code for a Java program resides in one or more "type" definitions, which are written in one or more source code files with `.java` extension. Don't be alarmed at the usage of the word "type" in the previous sentence. It is a technically more precise word than class and includes classes, interfaces, and enums. You will learn about interfaces and enums later but for now, we will just focus on classes.

2. **Java standard library classes:** As explained before, the Java platform comes with a huge amount of readymade components packaged in standard library classes. Any Java program can refer to those classes. You have already see the usage of one such readymade class - `java.util.ArrayList` - in the previous section. Since these classes are already compiled, a Java program doesn't need their source code to make use of them.
3. **Third party libraries:** Just like the Java standard library classes, a Java program may make use of components developed by other companies or developers. Such components are packaged in one of "jar" files. In this case also, the source code for those components may not be available. All you need to use those classes is to put the jar files of the library into the "classpath" of your program while compilation and execution.

Besides the above, a Java program may also include resource files such as images, properties, and configurations, which we will not be worrying about at this stage.

Structure of a Java class

The exam sometimes uses the phrase "Java program" to mean a Java source code file. However, this association is imprecise and is just a remnant from the procedural programming paradigm. From the exam perspective, you should know that a Java program primarily contains the following three things:

1. Zero or one package statement
2. Zero or more import statements
3. Zero or more class definitions (type definitions, to be more precise)

The order of the above three things is important. We will dig deeper into this in the next chapter. A class definition is composed of a class declaration and the class body, like this:

```
package com.abc; <--- OPTIONAL
import com.xyz; <--- OPTIONAL
class MyClass <--- class declaration
{ //everything from the opening { to the closing } is the class body
}
```

I will go into the details of a class declaration and definition later, but for now, just remember that a class declaration can contain the `public` modifier, an `extends` clause that specifies the name of the class that it extends and an `implements` clause that specifies the names of the interfaces that it implements. For example:

```
public class MyClass extends ParentClass implements Interface1, Interface2
```

The `public` modifier, and the `extends` and `implements` clauses are all optional. Although the JFCJA exam does not expect you to know much about the `extends` and `implements` clauses, it expects you to be aware of them.

3.4 Compiling and running a basic Java program

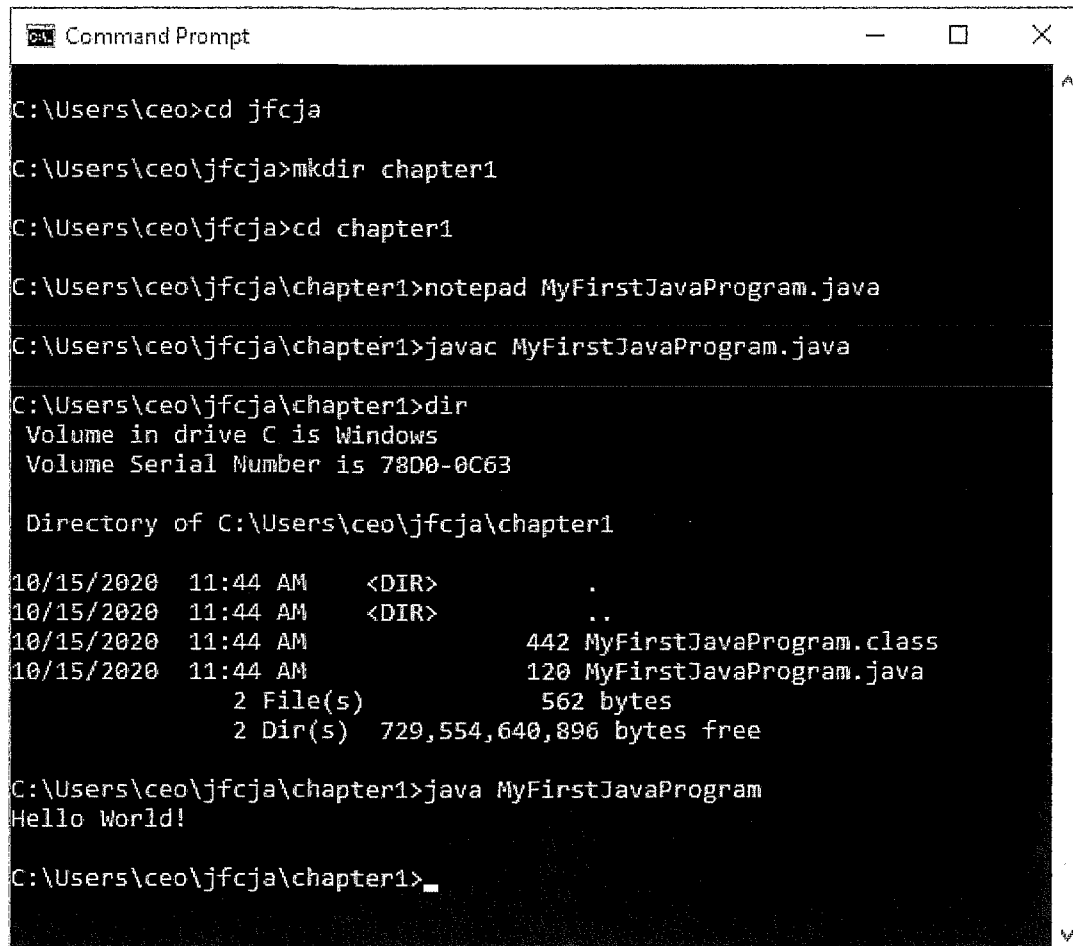
A specially written main method marks the entry point of a Java program. Thus, if the main method does not create any new object or does not call methods on other objects, the main method pretty much is the Java program. But Java being a highly object-oriented language, you can't just have a method without having a class. A method is the definition of a behavior and behavior can only be captured in a class. Thus, for creating even the most basic Java program that does nothing but print Hello World!, you must have a class! So, here it goes:

```
class MyFirstJavaProgram{
    public static void main(String[] args){
        System.out.println("Hello World!");
    }
}
```

Here are the steps that you can follow to compile and run the above program.

1. **Source code location:** Open up a command prompt and cd to the folder where you are going to keep your programs while going through this book. As I suggested earlier, you may create a folder named jfcja under C:\Users\<yourusername> \ to organize your all code. Under jfcja, create another folder called chapter1 for keeping all the code discussed in this chapter. If you are following this structure, then you should cd to C:\Users\ceo\jfcja\chapter1
2. **Create Java file:** Create a text file named MyFirstJavaProgram.java in the chapter1 folder and type up (or copy-paste, if you are reading the ebook version on your computer) the above code in this file and save it.
3. **Compile the source code:** Compile the Java file using the following command:
javac MyFirstJavaProgram.java
After compilation, you should see a new file named MyFirstJavaProgram.class created in the same folder.
4. **Run the program:** Run the program using the following command: java MyFirstJavaProgram
You should see Hello world! printed on the command line. Notice the absence of file extension .class while specifying the class name. To compile a Java source file, you must specify the extension of the file, i.e., .java though.

The following image shows the steps and the output that I got on my machine.



```
Command Prompt

C:\Users\ceo>cd jfcja
C:\Users\ceo\jfcja>mkdir chapter1
C:\Users\ceo\jfcja>cd chapter1
C:\Users\ceo\jfcja\chapter1>notepad MyFirstJavaProgram.java
C:\Users\ceo\jfcja\chapter1>javac MyFirstJavaProgram.java
C:\Users\ceo\jfcja\chapter1>dir
Volume in drive C is Windows
Volume Serial Number is 78D0-0C63

Directory of C:\Users\ceo\jfcja\chapter1

10/15/2020  11:44 AM    <DIR>          .
10/15/2020  11:44 AM    <DIR>          ..
10/15/2020  11:44 AM                442 MyFirstJavaProgram.class
10/15/2020  11:44 AM                120 MyFirstJavaProgram.java
                2 File(s)              562 bytes
                2 Dir(s)  729,554,640,896 bytes free

C:\Users\ceo\jfcja\chapter1>java MyFirstJavaProgram
Hello World!

C:\Users\ceo\jfcja\chapter1>
```

Figure 3.4.1: Compiling and running MyFirstJavaProgram.java

If you get any other output while compiling or running this program, make sure you have installed Java as per the instructions given in the previous sections. It is important to get this right and sort out any installation issues at this stage before moving forward.

Single file Java programs

Java designers felt that the two step compilation and execution of Java programs is too tedious when you are just trying to execute simple test programs. To make it simple, Java 11 allows you to directly execute a Java source file using the `java` command. For example, you can also execute the `MyFirstJavaProgram.java` file directly from the command line using the following command:

`java TestClass.java` It is certainly easier to run a Java program in one step but there are certain restriction with this approach, which I will not get into right now. It is best to stick to the two step execution involving separate compilation and execution steps at this stage.

Observe that this program does not have any package statement or import statements. You will see the usage of these statements soon once we proceed towards writing more complicated programs.

3.5 Understanding the main method

3.5.1 The main method

We discussed earlier that Java classes are not executables in the truest sense of the word. We cannot "execute" Java classes. The JVM is an executable. We execute the JVM. We pass the name (actually, the Fully Qualified Class Name, but more on that later) of the Java class that we want to run as an argument to the JVM. When the JVM runs, it locates the given class and looks for a specific method in that class. If it finds that method, it passes control to that method and this method then becomes the in-charge from there onward. If the JVM doesn't find that specific method, it errors out. In common parlance, we call it as executing or running a Java class or a Java program.

The method that the JVM is hardwired to look for in the class is called the "main" method and this method has a very specific signature - its name must be **main** and it must take exactly one parameter of type **String array**. In addition to this, it must return **void**, must be **public** and must also be **static**. It is free to declare any exception in its **throws** clause. If your class has such a method, the JVM can invoke this method and therefore, it is possible to execute the class. The meaning of String array, void, public, static, and throws will be clear as you proceed through the book but for now, just assume that this is how the main method has to be.

Examples of a valid main method:

1. `public static void main(String[] args){ }` - This is the version that you will see most of the time.
2. `public static void main(String... args){ }` - Note that `String...` is the same as `String[]` as far as the JVM is concerned. The three dots syntax is called `vargargs`. I will talk more about it later.
3. `public static void main(String args[]) throws Exception{ throw new Exception(); }` - The main method is allowed to throw any exception.

Examples of an invalid main method:

1. `static void main(String abc[]) { }` - Invalid because it is not public.
2. `public void main(String[] x){ }` - Invalid because it is not static.
3. `public static void main(String[] a, String b){ }` - Invalid because it doesn't take exactly one parameter of type String array.
4. `static void Main(String[] args){ }` - Invalid because it is not public and the name starts with a capital M. Remember that Java is case sensitive.

Note that all of the above methods are valid methods in their own right. It is not a compilation error if you have these methods in your class. But they cannot be accepted as the "main" method. JVM will complain if you try to execute a class on the basis of these methods. JVM has gotten smarter over the years and in Java 11, it gives out a very helpful error message that explains the problem with your main method. For example, if it is not static, you will see the following message:

```
Error: Main method is not static in class TestClass, please define the main method as:
public static void main(String[] args)
```

It is possible to have **overloaded** main methods in a Java class. I will talk about overloading in detail later, but for now, it means having multiple methods with same name but different parameters. The JVM looks for a specific main method as described above. All other main methods have no special meaning for the JVM.

Exam Tip

Many questions in the certification exam assume the presence of the main method. You may be given a code snippet and asked to determine the output. If you don't see any main method in the given code you need to assume that there is a main method somewhere that is invoked by the JVM and the given code or a method is invoked through that method.

3.5.2 Command line arguments

It is possible to provide any number of arguments while executing a class by specifying them right after the name of the class on the command line. The arguments must be separated by a space character. For example, if you want to pass three arguments to your class named TestClass, your command would be:

```
java TestClass a b c
```

The JVM passes on the arguments specified on the command line to the main method through its `String[]` parameter. In other words, the `String[]` parameter of the main method contains the arguments specified on the command line. An important implication of this is that all the arguments are passed to the main method as Strings. For example, if your command line is `java TestClass 1`, the main method will get a String array with one String element containing 1 and not an int 1.

Let me now present to you the following program to explain how to use command line arguments. This simple program prints the arguments that were passed to it from the command line.

```
public class TestClass{
    public static void main(String[] args) throws Exception{
        System.out.println(args.length);
        for(int i=0; i<args.length; i++){
            System.out.println("args["+i+"] = \""+args[i]+"\"");
        }
    }
}
```

The output of this program will tell you all you need to know about the command line arguments. The following is a table containing the command line used to execute the program and the

corresponding output generated by the program. The first line of the output shows the number of arguments received by the main method.

Command line used	Output	Inference
<code>java TestClass</code>		If no argument is specified, <code>args</code> contains a <code>String</code> array of length 0. Observe that a <code>NullPointerException</code> is not raised for <code>args.length</code> . That means <code>args</code> is not null . In this case <code>args</code> refers to a <code>String</code> array of length 0.
<code>java TestClass a</code>	<code>args[0] = "a"</code>	The first argument is stored at index 0. The first argument is NOT the name of the class.
<code>java TestClass a b c</code>	<code>args[0] = "a"</code> <code>args[1] = "b"</code> <code>args[2] = "c"</code>	Arguments can be separated by one or more than one white characters. All such separator characters are stripped away.
<code>java TestClass "a "</code> <code>b</code>	<code>args[0] = "a "</code> <code>args[1] = "b"</code>	If you put quotes around the value that you want to pass, everything inside the quotes is considered one parameter. Quotes are not considered as part of the argument. Observe that the first argument is <code>"a "</code> i.e. a <code>String</code> containing 'a' followed by a space.
<code>java TestClass "\""</code>	<code>args[0] = ""</code>	To pass a quote character as an argument, you have to escape it using a backslash.

Table 3.5.1: Arguments to `main()`

By the way, can you guess why the name of class is not passed in as an argument to the main method? Remember that unlike an executable program, you cannot change the name of a Java class file. The name of a Java class file will always be the same as the name given to the class in the Java source file. Therefore, the main method of a class always knows the name of its containing class.

3.5.3 The end of main

As discussed before, once the JVM passes control to the **main method** of the class that you are trying to execute, it is the main method that decides what to do next. As far as the JVM is concerned, your application has been "launched" upon invocation of the main method. In that sense, the main method is just an entry point of your application. So, what happens when the main method ends? Does the application end as well? Well, the answer is, it depends on what the main method does.

A simple Java program such as the one we used earlier to print arguments may have all its code in the main method. Once the main method ends, there is nothing else to do and so, the program ends. While you can write all your application code within the main method, Java applications are

usually composed of multiple classes. At run time, an application consists of instances of several classes that interact with each other by calling methods on each other. A Java application may also perform multiple activities in parallel, so, even if an activity implemented by one method ends, another activity implemented by some other method may still be going on. The code in `main` itself is just an activity. The end of the `main` method implies the end of only that activity. It doesn't mean the end of all the activities that may be going on in an application.

If it helps, you may think of your Java application as a fast food restaurant and the `main` method as its manager opening the restaurant in the morning. The restaurant need not close immediately after opening if there are no customers lined up. After opening the restaurant, the manager kicks off a lot of activities such as preparing the food, setting up the dining area, and waiting for customers. Such activities may continue side by side throughout the course of the day. When the last customer of the day is gone and when all such activities end, the restaurant closes for the day. The same thing happens in a Java application. The `main` method may kick off other activities that run side by side and the application ends only when all such activities come to an end.

Java allows executing activities in parallel using threads. This topic is beyond the scope of this exam so, I will not discuss it anymore in this book. But you should know that in a nutshell, an application doesn't end until all the threads started by the application, including the thread that executes `main`, end.

3.6 Object-Oriented Programming

3.6.1 A matter of perspective

A couple of years ago, while I was visiting India, I had a tough time plugging in my laptop charger in the 3-pin sockets. Even the international socket adapter kit, which had adapter pins of various sizes, was not of much help. Sometimes the receiver would be a bit too small and the pins wouldn't make steady contact or the pins would be a bit too wide and won't go into the receiver. I had to finally cut my cord and stick the bare copper wire ends directly into the sockets. I wondered, why do all these sockets in the same country have slight differences. During my stay, I observed that such minor variations were present in other things as well. Doors that wouldn't completely close, nuts that wouldn't turn properly, taps that wouldn't stop leaking, and other differences. Most of the time, people there take the trial and error approach when replacing parts. They work with the expectation that even if they get a part with the right size, it still may not fit perfectly. In other words, minor variations are expected and well tolerated.

This was unimaginable to me in the US, where everything just fits. I could buy a bolt from one shop and a nut from another, and it would work perfectly. Everything, from screws, nuts, and bolts, to wood panels, electrical parts, packing boxes, is standardized. One can easily replace a part with another built by a totally different company. You just have to specify the right "size".

This experience led me to a potential cause of why some OOP learners find OOP concepts confusing. Especially, beginners from non-western background find it really tough to grasp the fundamental concepts because they do not know the rationale behind so many rules of OOP. This is reflected in their application design.

In the US, and I imagine in other developed countries as well, things are extremely well defined. Products clearly specify how they should be used and in what cases they will fail. People can and do rely on these specifications because products work as expected and fail as defined. At the same time, people expect products to come with detailed specifications. Ready to assemble furniture is a prime example of how detailed these specifications can be. It's because of detailed and clear specifications that people feel comfortable in buying complicated ready-to-assemble furniture.

In short, people know exactly what they are getting when they acquire something. I think of it as the society being naturally "object-oriented".

Object orientation is just a name for the same natural sense of things fitting nicely with each other. A piece of code is not much different from the physical things I mentioned earlier. If you code it to a specification, it will fit just as nicely as your .16 inch nut on a .16 inch bolt, irrespective of who manufactured the nut and who manufactured the bolt. The point here is that the source of the concept of object-oriented programming is the physical world. If you want to grasp OOP really well, you have to start thinking of your piece of code as a physical component...a "thing" that has a well defined behavior and that can be replaced with another component that adheres to the same behavior. You would not be happy if you bought a tire that doesn't fit on your car's wheel even though you bought same 'size', would you? You should think about the same issues when you develop your software component. Someone somewhere is going to use it and they won't be happy if it fails at run time with an exception that you didn't say it would throw in a particular situation.

3.6.2 API

I talked about **Application Programming Interfaces** from a broader perspective in the first chapter. Let's dig into it a bit more because it connects my previous observation about relating programming concepts to real life. When you operate a switch do you really care about what exists inside the switch? Do you really care how it works? You just connect the switch to a light bulb and press it to switch the bulb on or off. It is the same with a car. A car provides you with a few controls that allow you to turn, accelerate, and brake. These controls are all you need to know how to drive a car.

You should think about developing **software components** in the same way. A software component doesn't necessarily mean a bunch of classes and packages bundled together in jar file. A software component could be as simple as a single class with just one method. But while developing even the smallest and the simplest of software components, you should think about how you expect the users to use it. You should think about various ways a user can use the component. You should also think about how the component should behave when a user doesn't use it the way it is expected to be used. In other words, you should specify an **interface** to your component, even before you start coding for it. Here, by interface, I do not mean it in the strict sense of a Java interface but a specification that details how to interact with your component. In a physical world, the user's manual of any appliance is basically its interface. In the software world, the specification of the publicly usable classes, methods, fields, enums, et cetera of a software component is its interface.

As an **application programmer**, if you want to use a component developed by someone else, you need to worry only about the interface of that component. You don't need to worry about what else it might contain and how it works. Hence the phrase 'Application Programming Interface'.

In the Java world, a collection of classes supplied by a provider for a particular purpose is called a **library** and the **JavaDoc** documentation for those classes describes its API. As discussed before, when you install the **Java Runtime Environment (JRE)**, it includes a huge number of classes and packages. These are collectively called the standard Java library and the collection of its public classes and interfaces is called the **Java API**.

The Java API contains a huge amount of ready-made components for doing basic programming activities such as file manipulation, data structures, networking, dates, and formatting. When you write a Java program, you actually build upon the Java API to achieve your goal. Instead of writing all the logic of your application from scratch, you make use of the functionality already provided to you, free of cost, by the Java library and only write code that is specific to your needs. This saves a lot of time and effort. Therefore, a basic understanding of the Java API is very important for a Java programmer. You don't need to know by heart all the classes and their methods. It is practically impossible to know them all, to be honest. But you should have a broad idea about what the Java API provides at a high level so that when the need arises, you know where to look for the details. For example, you should know that the standard Java library contains a lot of classes for manipulating files. Now, when you actually need to manipulate a file, you should be able to go through the relevant Java packages and find a Java class that can help you do what you want to do.

The **JFCJA** exam requires that you know about some of the widely used packages and classes of the Java API. I will go through them in detail in this book.

If you keep the above discussion in mind, I believe it will be very easy for you to grasp the concepts that I am going to talk about throughout this book.

3.6.3 Principles of OOP

Object-Oriented Programming is a programming paradigm where a software application is visualized and designed as a bunch of objects interacting with each other. OOP is inspired by how real life objects behave and interact. If you look around, everything that you see is some kind of an object. Your TV, remote, chair, pen, computer, book, fan are all objects that exhibit certain properties and also provide ways for other objects to interact with them. In that sense, the whole world around you, including yourself, is just a bunch of objects interacting with each other.

Similarly, when developing a software program for solving particular problem in the OOP paradigm, you identify objects in that domain and the ways in which they interact with each other. In Java, these objects are modeled using classes and their behavior is modeled as methods of those classes. The interaction between the objects causes changes to the state of the objects, which is represented using the data fields of the objects.

Let me give you a quick example. Let's say you are developing an application for managing the inventory of a car dealership. This application is supposed to maintain a list of cars. Here, you can observe two type of objects at the outset - Dealership and Car. Furthermore, you know that there is only one dealership but the dealership has many cars. Thus, there should be just one Dealership object and multiple Car objects in the system.

Dealership should have a way to add, remove, and display the cars that are currently present in the inventory. These operations can be modeled as methods of Dealership. The data that the Dealership object holds at any moment, that is, the list of Car objects, is the state of the Dealership object. An interaction with the Dealership object may modify its state. For example, adding and removing a Car from the Dealership will modify its list of cars. It is important to understand here that software objects are designed to exhibit only such behavior as is relevant for the application. A real dealership does a thousand different things but they are not relevant for the inventory management application. Since we are modeling a dealership from inventory management perspective, we abstract out only those operations that are important for managing the inventory. That is why, our Dealership object may support only a few operations. Similarly, our Car objects may exhibit only a few properties such as the make, model, and year. This process of identifying relevant behavior for designing software objects is called **abstraction** in OOP terminology.

The above discussion illustrates the fundamental thought process involved in designing a solution using the OOP paradigm. Some people find OOP less intuitive at the beginning because there are no physical objects in software. Modeling software parts as objects containing code as well as data requires abstract thinking. However, it does payoff in the long run. In contrast, there is the "Procedural Programming" paradigm, which perceives software as a series of processes executed over a given set of data. An application is broken up into various processes which call one another as and when required. For example, in the car dealership example, you could have "show cars", "add car", "remove car" as procedures that manipulate a list of cars stored somewhere. Beginners typically find procedural programming a lot easier to comprehend but experience has shown that OOP is more suitable for developing modern applications.

Over the years, the process of modeling software objects has evolved and several strategies and practices have been established to make this process more efficient. Discussing all the details of OOP will require a book of its own. So, here, I will only cover the four important principles of OOP that you should keep in mind. Do not worry if you do not understand or appreciate them fully at this stage. You will get the hang of OOP slowly as you write more and more programs. In fact, one of the reasons why Java has been so successful is that it makes OOP easier to implement. You will gradually realize that Java has features that prevents programmers from veering off towards procedural programming and that gently steer them towards OOP.

Abstraction

Abstraction is the process of capturing relevant details of an entity or a concept. In Java, such an entity is generally described using the "class" artifact. For example, in the car dealership example above, the concepts of dealership and car can be described using a Dealership class and a Car class. Once the concept is defined, multiple instances of the concept can be created.

Encapsulation

Encapsulation means to hide the implementation details of an entity. An entity should only expose the "what" and not the "how". For example, anyone interacting with a car dealership only needs to be able to browse all the cars that the dealership has, not how the dealership maintains its cars. As long as the dealership is able to provide a way to add/remove/display cars to the users, it

shouldn't matter if the dealership changes the way it manages its inventory. Hiding implementation details ensures that various components of an application do not unwittingly depend on each other's implementation details. This allows the components to evolve without breaking the system. In Java, encapsulation is typically achieved by using appropriate access modifiers for the members of a class. Typically, data members of a class are declared private while methods are declared public.

Inheritance

Inheritance is process of creating specialized entities by extending existing entities. It is also seen as establishing an "is a" or a "parent child" relationship between two entities. Inheritance provides a way to add new entities or new features to the system quickly by reusing existing ones. For example, the inventory management system for the car dealership may want to display Sports cars differently while still treating them as regular cars in other respects. One could, therefore, create a specialized entity named SportsCar that extends Car. By extending Car, a SportsCar would automatically inherit all the features and functionality of a Car.

Polymorphism

Polymorphism means same entity exhibiting different behavior depending on the context. Polymorphism is actually a direct consequence of inheritance. For example, when you want to count the number of cars in a dealership, it doesn't matter whether a car is a sports car or just a regular car. But when you feel the difference when you drive a sports car. So, while counting, all the cars behave in the same way, but while driving, a sports car behaves differently from other cars.

3.6.4 OOP in Java

Java is an Object-Oriented Programming language. What it means is that the Java programming language provides constructs that help you implement your application in the OOP way. It doesn't mean that Java allows applications to be developed only in the OOP way. Some programmers, who are more conversant with procedural programming, think in terms of functions and procedures and end up writing their programs in that manner. That is not the right approach though. In fact, you will realize later that Java makes it harder for you to write your programs in the procedural way. If you are going to be writing programs in Java, you should start thinking in terms of objects and their interactions instead of functions calling each other. A Java program or a Java application is, therefore, nothing but a bunch of objects interacting with each other.

As explained before, to create objects, you first need to describe the relevant details of those objects using the process called abstraction. The most basic building block to do that in Java is called a "class".

Describing Objects

A class allows you to describe the data elements and the behavior of the objects that you want to deal with in your application. In OOP parlance, data elements are called "attributes" or "properties" and a behavior is called a "method".

For example, the inventory management application that we talked about earlier has two kinds of objects - Dealership and Car. The details of these two can be captured using two classes as follows:

```
class Car{
    String make; String model; int year;
}
class Dealership{
    java.util.ArrayList cars = new java.util.ArrayList();
    void addCar(Car c){ cars.add(c); }
    void removeCar(Car c){ cars.remove(c); }
    void printCars(){
        for(Car c : cars){
            System.out.println("Make: "+c.make+" Model: "+c.model+" Year: "+c.year);
        }
    }
}
```

The Car class captures the details of a car. At this time, we care only about the make, model, and year of a car. The Dealership class captures the details of the dealership. It has an attribute named cars, which is supposed to contain the list of available cars and a few methods to manage and display that list. Observe that these Car and Dealership are classes, not objects. They merely describe what a car and a dealership objects contain and how they behave. To make a program, we will have to create objects using these classes and make them interact.

Describing Behavior

Let me ask you a question, do you really care about the circuitry that is involved when you switch things on and off? No, I guess. As long as you see a switch that switches a device on or off, you are good to go, right? You simply push the switch on or off, and it is up to the device connected to that switch to perform the switching.

What we have just identified is a behavioral aspect of things. If we say that something switches on or off, we are essentially describing a behavior of that thing. For example, a Fan, a LightBulb, and a Car can all be switched on or off. Here, "switchable" is the behavior that is exhibited by the Fan, LightBulb, and Car objects.

It is the same with Java objects as well. Once we identify a particular behavior of a range of objects, we can deal with all of them in the same manner without worrying about what those objects are or how they implement that behavior. Java allows us to capture just the behavior using an "interface". For example,

```
interface Switchable{
    void on();
    void off();
}
```

The above is merely a description of the behavior. To us, anything that can be switched on or off is a Switchable. The important thing to understand here is that we are not actually trying to switch anything on or off because we don't even know how different things do this activity. Indeed, a Fan and a Car switch on and off differently. In other words, there are no implementation details in an interface.

It is the job of the class of objects to decide how to implement a particular behavior as is appropriate for them. In Java, we tie the behavior to a class using the keyword "implements". For example,

```
class Fan implements Switchable{
    boolean on;
    void on(){
        //perform all the things that are needed to switch it on
        on = true;
    };
    void off(){
        //perform all the things that are needed to switch it off
        on = false;
    };
}
```

The benefit of abstracting out just the behavior using interfaces is that they making the interaction between different objects a lot easier to maintain. Think of it this way - a care taker of a hotel can be directed to switch on/off anything as long as it is has the on/off switch. What if different devices had different ways to turn them on or off? People would go crazy learning new ways to deal with their devices. Similarly, in programming, it is important to identify and abstract out behavior of the objects in your application as much as possible.

The JFCJA exam **does not** expect you to know about interfaces but I have discussed it in brief because some of the topics covered in JFCJA do require you to be aware of the concept.

Creating Objects

Once you have described the kind of objects you want to deal with, you can create objects at will using "new". For example, Car car1 = new Car(); or Dealership dealer = new Dealership(); You might have noticed new java.util.ArrayList() in the Dealership class. Yes, java.util.ArrayList is a class and new java.util.ArrayList(); creates an ArrayList object. We are using it to store the car objects. It represents the current inventory of the dealership. Don't worry about where this class came from or about the java.util part right now.

Making Objects interact

Objects interact with each other by calling methods on or accessing data fields of each other. For example, the showCars method of the Dealership class accesses the make, model, and year, attributes of a Car object by using a "reference" to a Car object.

Launching a Java program

I said above that a Java program is nothing but a bunch of objects interacting with each other. But who creates those objects? Who initiates the interaction among those objects? It is like all the players of the game are on the field but who starts the play? In Java, we do that by creating special method called "main". We launch the Java program by asking the JVM to execute this main method. For example:

```
class MyApplication{
    public static void main(String[] args){
        Dealership dealer = new Dealership();
        Car c = new Car();
        dealer.addCar(c);
        dealer.showCars();
    }
}
```

Don't worry about all the details such as `public`, `static`, `void`, and `String[]` that you see in code above. We will get to all that later. Just observe that we are creating a few objects (a `Dealership` object and a `Car` object) and calling a few methods (`addCar` and `showCars`). The ball is set rolling when the JVM executes the main method. For this reason, the main method is also called the "entry point" of a Java application.

When objects start interacting in an application, they may create even more objects as governed by the code written in those classes. This is basically what any Java program does.

It is important to understand where exactly are the objects are created and what do we mean by "reference". I will explain it with a detailed example now.

3.6.5 Relation between Class, Object, and Reference

A **class** is a template using which **objects** are created. In other words, an object is an instance of a class. A class defines what the actual object will contain once created. You can think of a class as a cookie cutter. Just as you create cookies out of dough using a cookie cutter, you create objects out of memory space using a class. When you use the "new" keyword, you create a new object of that class.

To access an object, you need to know exactly where that object resides in memory. In other words, you need to know the "address" of an object. Once you know the address, you can call methods or access fields of that object. It is this "address" that is stored in a reference variable.

If you have trouble understanding this concept, try to imagine the relationship between a Television (TV) and a Remote. The TV is the object and the Remote is the reference variable pointing to that object. Just like you operate the TV using the remote, you operate on an object using a reference pointing to that object. Notice that I did not say, "you operate on an object using its reference". That's because an object doesn't have any special reference associated with it. Just as a TV can have multiple remotes, an object can have any number of references pointing to it. One reference is as good as any other for the purpose of accessing that object. There is no difference

between two references pointing to the same object except that they are two different references. In other words, they are mutually interchangeable.

Now, think about what happens when the batteries of a remote die. Does that mean the TV stops working? No, right? Does that mean the other remote stops working? Of course not! Similarly, if you lose one reference to an object, the object is still there and you can use another reference, if you have it, to access that object.

What happens when you take one remote to another room for operating another TV? Does it mean the other remote stops controlling the other TV? No, right? Similarly, if you change one reference to point to some other object, that doesn't change other references pointing to that object. The following picture illustrates the situation:

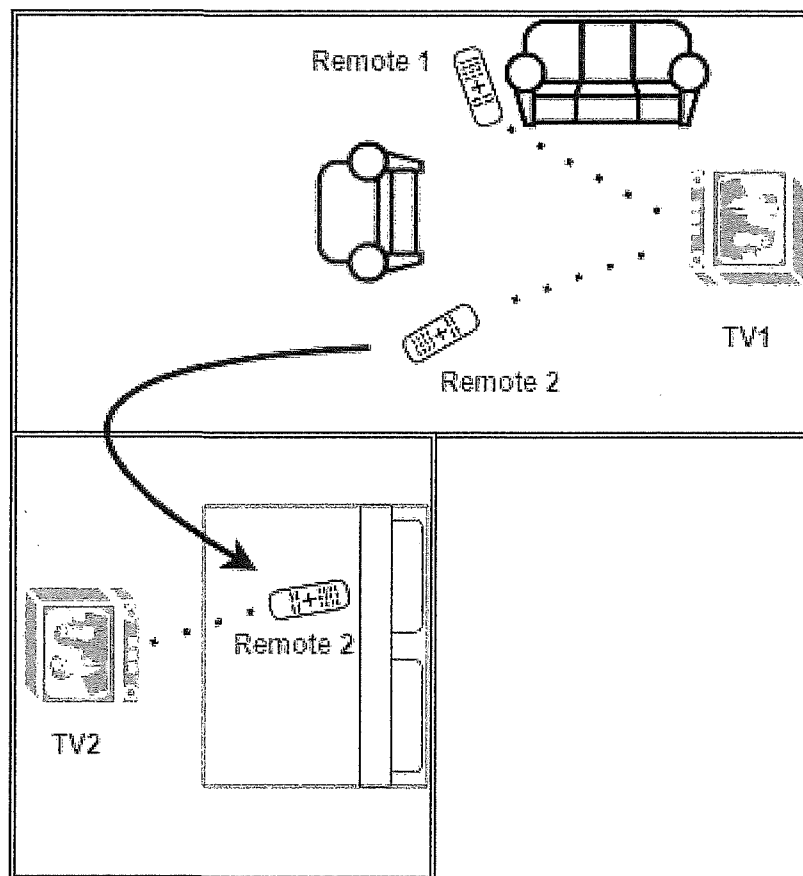


Figure 3.6.1: Relation between Class, Object, and Reference

Let me now move to an example that is closer to the programming world. Let's say, you have the following code:

```
String str = "hello";
```

"hello" is the actual object that resides somewhere in the program's memory. Here, `str` is the remote and "hello" is the TV. You can use `str` to invoke methods on the "hello" object.

A program's memory can be thought of as a long array of bytes starting with 0 to NNNN, where NNNN is the location of last byte of the array. Let's say, within this memory, the object "hello"

resides at the memory location 2222. Therefore, the variable `str` actually contains just 2222. It doesn't contain "hello". It is no different from an `int` variable that contains 2222 in that sense.

But there is a fundamental difference in the way Java treats **reference variables** and non-reference variables (aka **primitive variables**). If you print an `int` variable containing 2222, you will see 2222 printed. However, if you try to print the value `str`, you won't see 2222. You will see "hello". This is because the JVM knows that `str` is defined as a reference variable and it needs to use the value contained in this variable to go to the memory location and do whatever you want to do with the object present at that location. In case of an `int` (or any other primitive variable), the JVM just uses the value contained in the variable as it is. Since this is an important concept, let me give you another example to visualize it. Let us say Paul has been given 2222 dollars and Robert has been given bank locker number 2222. Observe that both Paul and Robert have the same number but Paul's number denotes actual money in his hands while Robert doesn't have actual money at all. Robert has an address of the location that has money. Thus, Paul is like a primitive variable while Robert is like a reference variable.

Another important point is that you cannot make a reference variable point to a memory location directly. For example, you can set the `int` variable to 2250 but you can't do that to `str`, i.e., you can't do `str = 2250`. It will not compile. You can set `str` to another string and if that new string resides at a memory location 2250, `str` will indeed contain 2250 but you can't just store the address of any memory location yourself in any reference variable.

As a matter of fact, there is no way in Java to see and manipulate the exact value contained in a reference variable. You can do that in C/C++ but not in Java because Java designers decided not to allow messing with the memory directly.

You can have as many references to an object as you want. When you assign one reference to another, you basically just copy the value contained in one reference into another. For example, if you do `String str2 = str`; you are just copying 2222 into `str2`. Understand that you are not copying "hello" into `str2`. There is only one string containing "hello" but two reference variables referring to it. Figure 1 illustrates this more clearly.

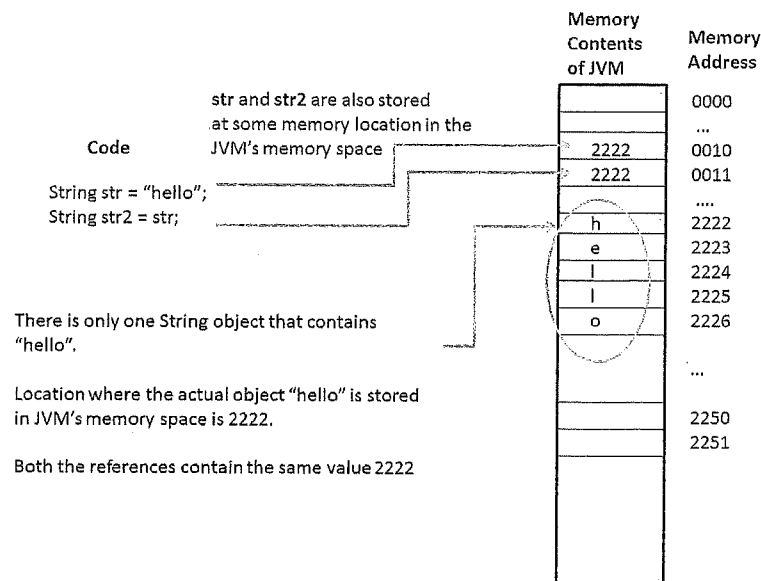


Figure 3.6.2: Object and Reference

If you later do `str = "goodbye";` you will just be changing `str` to point to a different string object. It does not affect `str2`. `str2` will still point to the string "hello".

The question that should pop into your head now is what would a reference variable contain if it is not pointing at any object? In Java, such a variable is said to be `null`. After all, as discussed above, a reference variable is no different from a primitive variable in terms of what it contains. Both contain a number. Therefore, it is entirely possible that a reference that is not pointing to any object may actually contain the value 0. However, it would be wrong to say so, because a reference variable is interpreted differently by the JVM. A particular implementation of JVM may even store a value of -1 in the reference variable if it does not point to any object. For this reason, a reference variable that does not point to any object is just `null`. At the same time, a primitive variable can never be `null` because the JVM knows that a primitive variable can never refer to an object. It contains a value that is to be interpreted as it is. Therefore,

```
String str = null; //Okay
int n = 0; //Okay
```

```
String str = 0; //will not compile
int n = null; //will not compile.
```

3.6.6 static and instance

We saw previously how we use a class to abstract an entity and how we instantiate objects of that class to create a program or an application. I also mentioned earlier that it is possible to write programs in Java using the procedural programming paradigm, in which we think of tasks or routines that can be invoked to achieve our goal. For example, if we want to compute simple interest for a given principle, rate, and time, we would want to simply invoke a procedure named

`computeSimpleInterest` with three arguments. It would compute the interest and return the value. Basically, we are not interested in abstracting any entity here. We are just interested in writing a procedure or a function to compute the simple interest for a given set of inputs. There is no need for objects or the interplay of objects in this situation. Since, we don't need any object here, ideally, we shouldn't need any class either because the whole purpose of a class is to create objects!

However, unlike in other languages such as C/C++/Python, you can't just have independent procedures or functions in Java. Every piece of code or every set of instructions must belong to one or another class. This may seem like a big restriction to you at first but it is not. Java has a slightly roundabout way of achieving the same by using the "**static**" keyword.

In English, the word static means something that doesn't change or move. From that perspective, it is a misnomer. Java has a different word for something that doesn't change - **final**. I will talk more about final later. In Java, static means something that belongs to a class instead of belonging to an instance of that class. If it helps, you can think of it as something that remains the same for all instances of a class. The idea here is that if we use something that belongs to the class instead of an object of the class, we won't need to create an object of that class. Let me show you how this works by creating a function to compute simple interest as a static method - I am calling it "method" instead of "function" because method is the right terminology in the Java world - of a class:

```
class FinancialFunctions{
    static double computeSimpleInterest(double p, double r, double t){
        return p*r*t/100;
    }
}
class TestClass{
    public static void main(String[] args){
        double interest = FinancialFunctions.computeSimpleInterest(1000, 10, 1);
        System.out.println(interest);
    }
}
```

You can copy paste the above program in a file named `TestClass.java` and run it. Observe that our main method uses the `computeSimpleInterest` method without creating any object of the `FinancialFunctions` class. There is no usage of the `new` keyword anywhere in the above program.

Just like static methods, you can have static variables in a class. As we discussed earlier, a class is just a template. You can instantiate a class as many times as you want and every time you instantiate a class you create an instance of that class. Now, recall our cookie cutter analogy here. If a class is the cookie cutter, the fields defined in the class are its patterns. Each instance of that class is then the cookie and each field will be imprinted on the cookie - except the fields defined as static. In that sense, a static member is kind of a tag stuck to a cookie cutter. It doesn't apply to the instances. It stays only with the class. That is why, static variables are also called "class variables".

The following partial code explains how it works:

```
class Account {
    String accountNumber;
    static int numberOfAccounts;
}
...

//Create a new Account instance
Account acct1 = new Account();

//This Account instance has its own accountNumber field
acct1.accountNumber = "A1";

//But the numberOfAccounts fields does not belong to the instance, it belongs to the
    Account class
Account.numberOfAccounts = Account.numberOfAccounts + 1;

//Create another Account instance
Account acct2 = new Account();

//This instance has its own accountNumber field
acct2.accountNumber = "A2";

//the following line accesses the same class field and therefore, numberOfAccounts is
    incremented to 2
Account.numberOfAccounts = Account.numberOfAccounts + 1;
```

Important points about static -

1. static is considered a non object-oriented feature because as you can see in the above code, static fields do not belong to an object.
2. Here is a zinger from Java designers - even though static fields belong to a class and should be accessed through the name of the class, for example, `Account.numberOfAccounts`, it is not an error if you access it through a variable of that class, i.e., `acct1.numberOfAccounts`. Accessing it this way doesn't change its behavior. It is still static and belongs to the class. Therefore, `acct2.numberOfAccounts` will also refer to the same field as `acct1.numberOfAccounts`. This style only causes confusion and is therefore, strongly discouraged. Don't write such code. Ideally, they should have disallowed this usage with a compilation error.
3. Just like fields, methods can be static as well. A static method belongs to the class and can be accessed either using the name of the class or through a variable of that class.
4. The opposite of static is instance. There is no keyword by that name though. If a class member is not defined as static, it is an instance member.

The purpose of using static for defining the main method should also be clear now. When we run our program, the JVM doesn't have to create an object of our class. It simply invokes the main method of the class whose name we pass as an argument on the command line.

3.7 Exercise

1. Run `java -help` and `javac -help` commands on the command line. Observe the output.
2. How important is the name of the file of a Java source code? What are the advantages and disadvantage of writing a class in a file with the same name?
3. What should be the name of a file that contains a class with the main method?
4. In their zeal to make their class as useful and functional as possible, a developer has created the following class:

```
class DoEverything{
    int INTERESTRATE = 10;
    double computeInterest(double p, double t){
        ...
    }
    String defaultFilePath
    double saveDataToFile(String data){
        ...
    }
}
```

Which OOP principles does this class violate and why?

5. Which part of a class does encapsulation try to control - data or functionality? Why?
6. Information hiding is a key principle of OOP. What exactly does it try to hide when applied to a Java class?
7. Think of different ways of code reuse. (Hint: Inheritance and Aggregation) Which one falls under OOP?